항공기 날개 공력탄성학 및 플러터 진동 해석

Aeroelasticity and Flutter Analysis of Aircraft Wings

과목번호 120520-1

학번 C417025

이름 김세경

제출일 June 22, 2026

1 시스템 모델 및 상태공간,전달함수 정의

1.1 모델 간소화 및 자유도 축소

해석의 용이성과 자원의 한계를 고려하여 다음 가정 하에 진행하였다. 날개는 형상이 변하지 않는 대칭,균일 구조이다; 이때 탄성에 의한 효과는 길이 방향만 고려하며, 날개의 길이는 무한하여 wing vortex와 같은 3차원 효과는 무시한다;길이 방향 탄성효과는 익형에 달린 두 스프링과 두 댐퍼로 근사한다.(에어포일 형상은 탄성의 영향을 받지 않는 강체로 가정한다.) 실험은 풍동 내부를 가정하며 실제 날개와의 유사성은 Dynamic Similarity로 보장된다. 순항고도와 같이 비교적 steady한 상황의 미소 섭동(perturbation)에서 시스템은 선형시간불변(LTI)으로 간주 될 수 있다. [4], 마지막으로,난류(gust)는 구조적 스트레스가 지배적인 수직 방향만 고려한다.

1.2 모델

날개의 거동학을 알기 위해 풍동 안에서 실험이 이루어진다고 하면, 날개의 익형(airfoil)이 일정하기 때문에 Chord length c_{chord}는 상수다. 그리고 무한히 긴 날개에서, 동체에서 b만큼 떨어진 날개의 단면에 대한 등가모형을 아래와 같은 *strut-supported* wind tunnel model로 가정할 수 있다.[2]

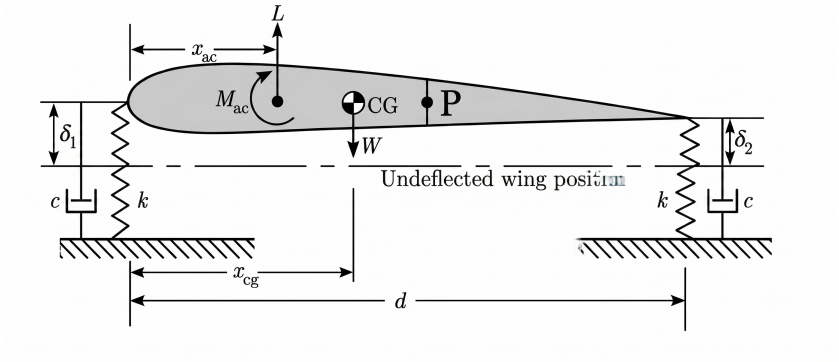


Figure 1: Cross section of strut-supported wind-tunnel model.
The wing is represented as a rigid airfoil on two spring-damper support

동일한 강성을 가진 스프링 k가 d만큼 떨어진 상태로 두개 위치하며, 각각의 스프링 옆에는 동일한 감쇠계수 c를 가진 감쇠기가 위치한다. 익형의 중간지점 P에 대해서,아래와 같이 운동을 기술하기 위한 두개의 변수를 정의한다.

h — the *plunge*, 기준점 *undeflected wing position*에 대한 수직거리 (positive downward)

θ — the *pitch*, 익형의 회전 (positive nose-up).

임의의 chordwise location x에 대한 vertical displacement z는 아래와 같이 정의할 수 있다.

$$z = h + x\theta \tag{1}$$

해당 기준점 P에 대하여, 익형의 시작점을 기준으로 위 그림과 같이 공력중심,무게중심,중점 순으로 위치하며,기준점 P에 대하여 P와 무게중심까지의 거리를 x_{cg}^p로, 공력중심까지의 거리를 x_{ac}^p로 정의한다.

1.3 운동방정식 유도

약 3 4주차에 걸쳐서 아래의 운동방정식을 유도하였다. 질량 $\mathbf{M}$,구조 감쇠 $\mathbf{C}$, 구조 강성 $\mathbf{K}_s$는 라그랑지안에서, 공력 감쇠 $\mathbf{C}_a$, 공력 강성 $\mathbf{K}_a$, 돌풍 입력 벡터 $\mathbf{F}_g$는 준-정상상태[1] 공력 모델로부터 얻어진다. 각 행렬의 자세한 유도과정은 부록 appendix A에 수록하였다.

$$\boxed{\mathbf{M}\ddot{\mathbf{q}} + \left(\mathbf{C} + \mathbf{C}_a\right)\dot{\mathbf{q}} + \left(\mathbf{K}_s + \mathbf{K}_a\right)\mathbf{q} = \mathbf{F}_g\, w_g(t).} \tag{2}$$

여기서 $\mathbf{q} = [h,\ \theta]^\top$는 plunge와 pitch를 모은 일반화 좌표이고, $w_g(t)$는 수직 돌풍 성분이다.

1.4 상태공간 정의

(2)은 $\mathbf{q}$에 대한 2차 미분방정식이므로, 상태벡터(시스템벡터)를 $\mathbf{x} = [\mathbf{q},\ \dot{\mathbf{q}}]^\top \in \mathbb{R}^4$로 정의하면,위의 운동방정식은 상태공간으로 나타내 줄 수 있다. $\mathbf{K}_{\text{eff}} = \mathbf{K}_s + \mathbf{K}_a$, $\mathbf{D}_{\text{tot}} = \mathbf{C} + \mathbf{C}_a$로 두고 $\ddot{\mathbf{q}}$에 대해 정리한 뒤, 출력을 $\mathbf{y} = \mathbf{q} = [h,\ \theta]^\top$라 하면,

$$\boxed{\dot{\mathbf{x}} = \mathbf{A}\,\mathbf{x} + \mathbf{B}\,w_g, \qquad \mathbf{y} = \mathbf{C}_{\text{out}}\,\mathbf{x}} \tag{3}$$

를 얻는다. 행렬 $\mathbf{A}, \mathbf{B}, \mathbf{C}_{\text{out}}$의 구체적 형태와 변환 과정은 부록 appendix A.7에 정리하였다. 상태공간을 앎으로써 임의의 돌풍 입력 w_g에 대한 익형의 변위 · 회전 응답을 예측할 수 있고, 시스템 행렬 $\mathbf{A}$의 고유값 · 고유벡터로부터 안정성, 공진주파수, 모드형태를 분석할 수 있다.

[1] quasi-steady

1.4.1 모델 계수

여러 소스들[1, 4, 2]과 FAA에 공시되어져 있는 자료들을 통해서 NACA2412의 데이터를 혼합하면, 아래와 같은 풍동에서 실험한 데이터를 얻을 수 있다. 해당 계수를 베이스로 하여금 앞으로 분석을 진행할 것이다.

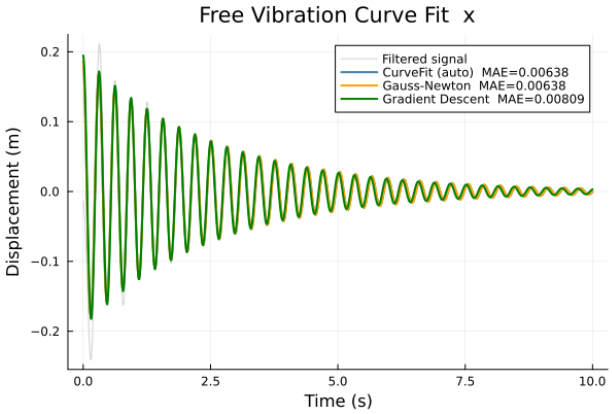
Table 1: Illustrative parameters (chosen to give two well-separated peaks).

Symbol	Value	Symbol	Value
m	$1.50\,\mathrm{kg}$	ρ	$1.23\,\mathrm{kg\,m^{-3}}$
I_p	$0.008\,40\,\mathrm{kg\,m^2}$	U	$10.0\,\mathrm{m\,s^{-1}}$
S	$0.0225\,\mathrm{kg\,m}$	c_{chord}	$0.300\,\mathrm{m}$
k (each)	$900\,\mathrm{N\,m^{-1}}$	b_{span}	$0.500\,\mathrm{m}$
c (each)	$1.00\,\mathrm{N\,s\,m^{-1}}$	$C_{l\alpha}$	$6.00\,\mathrm{rad^{-1}}$
d	$0.220\,\mathrm{m}$	e_a	$0.0450\,\mathrm{m}$

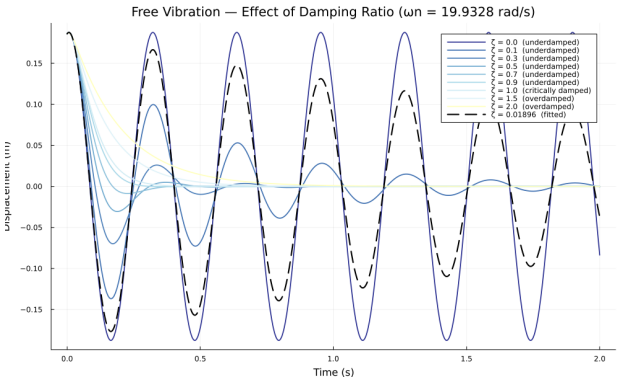
2 주차별 스토리텔링

2.1 curve-fitting을 이용한 등가모델(WEEK1)

1주차에서 우리는 날개를 1DOF mass-damp-spring 시스템[2]으로 설계해보았다.센서 노이즈를 제거한 후, 여러 curve-fitting 알고리즘(Gradient Descent,Gauss-Newton etc)등을 통해 수치해석적 기법을 통해, 적당한 수준으로 모델링이 이루어졌을 때, 그에 상응하는 여러 진동학적 상수를 얻을 수 있음을 알 수 있었다.fig. 2 감쇠비가 1 이하이면 진폭이



(a) 여러 curve fitting algorithm을 이용한 등가모델 찾기
X = 0.18813,ζ=0.01896,w_n = 19.9328, ϕ=0.1525



(b) damping ratio ζ 변화에 따른 진동 소산 현상의 변화.

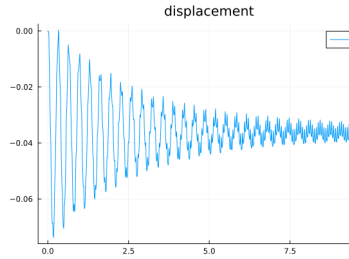
Figure 2: $x(t) = Xe^{-\zeta w_n t}\cos(\sqrt{1-\zeta^2}w_n t - \phi)$

크고,1이상이면 진동소산까지의 시간이 오래걸림을 알 수 있다. 이 과정에서 감쇠비(ζ=1)근처로 디자인을 해야함을 알 수 있다. 자재에 따라서 등가 구조 강성 k가 바뀌면 자연주파수 $w_n = \sqrt{k/m}$이 증가하므로 진동을 더 빠르게 할것이며, 이는 동일한 ζ에 대해서 $w_d = \sqrt{1-\zeta^2}w_n$이므로, 결과론적으로 transient state(초기 단계)에서 더 빠른 진동을 일으킬 것임을 알 수 있다.

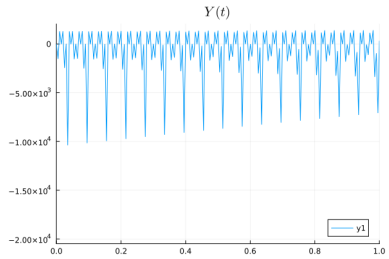
2.2 가진과 질량 강성의 관계(WEEK2)

2주차에서는 1주차에서 모델링된 모델을 통해 임의의 가진에 대한 시스템의 반응, 그리고 질량 변화에 따른 시스템 거동의 변화에 대해 살펴 보았다. 아래와 같이 일반적인 소형항공기에서 많이 쓰이는 Rotary engine에 대한 진동에 의한 힘이 시스템에 전달 될 때 반응하는 거동을 살펴보면, 아래와 같이, 초반(transient)구간에서는 흥분현상으로 인해 큰 진폭을 보였으나, 갈수록 정상상태로 진입하여, 엔진의 진동수와 일치해가는 현상을 발견할 수 있었다. [3] 또한, 스프링 강성이 감소함에 따라, 1DOF의 고유주파수식, $w_n = \sqrt{k/m}$에 따라, 고유주기가 증가하는 모습을 볼 수 있었으며, 진동의 진폭또한 증가하는 것을 확인 할 수 있었다. 진폭이 증가하는 현상은 질량이 증가함에 따라 어느정도 감소하는 것을 알 수 있었다.이러한 관계(날개의 질량을 과도하게 줄이면 강성또한 낮아지며 진동으로 인한 피로가 증가함)에서, 질량과 강성 사이에 최적의 경우를 찾아야 함을 알 수 있었고, 그것을 찾기 위해 Pareto frontier와 같은 최적화 기술이 쓰임을 알 수 있었다. 위에 보이는 사진들과 같이, w_n을 고정시키고 탐구를 해보면, 질량을 증가시키면 시스템의 진폭이 감소하나, 이는 강성이 그만큼 증가해야한다는 것을 의미한다는 것을 알 수 있다.

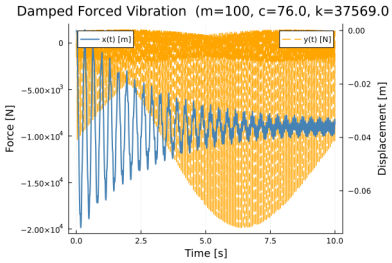
[2] $m\ddot{x} + c\dot{x} + kx = F$



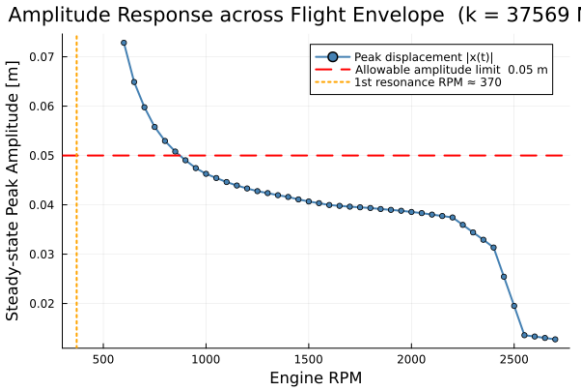
(a) Displacement



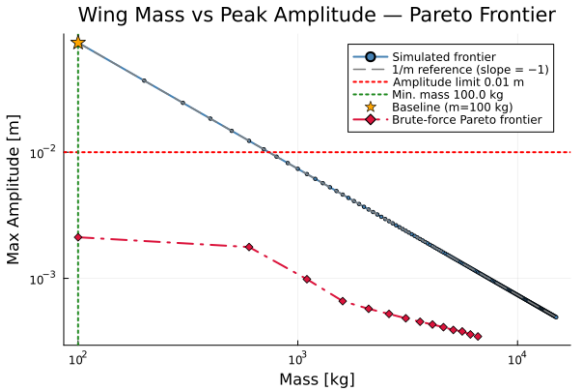
(b) Force



(c) Total



(d) RPM에 따른 공진 응답



(e) 질량-강성 Pareto frontier

Figure 3: 2주차 결과

이러한 관계 속에서 항공기의 연료가 탱크가 아닌 날개에 먼저 저장되는 이유도 경량화의 목적을 이루면서 날개쪽에 질량을 늘려, 진동으로 피로를 줄이기 위함임을 유추 할 수 있었다. 마지막으로, 탐구 과정 도중, 위의 1DOF 모델은 아래 그림과 같이 상대적 저주파에서 공진현상을 보임을 알 수 있었다. 이로써 엔진의 과도한 idling이 연비는 아낄 수 있지만, 기체에 가하는 피로로 인해 득보다 실이 더 클 수도 있다는 것을 알게 되었다.

2.3 날개의 로우패스 필터 특성과 공명현상(WEEK3)

3주차부터는 위에서 서술한 2DOF 모델을 활용하여 탐구를 진행하였다. 난기류는 그 모델에 따라 *von Kármán,Dryden* turbulent model 등 다양한 모델이 존재한다.[3] 이러한 모델에서 알 수 있는 특성으로는, 난기류는 저주파 부분에서 큰 진폭을 가지며,고주파 부분은 거의 존재하지 않는다는 부분을 확인 할 수 있었다. 따라서 해당 근거에 따라서,아래 그림과 같이 날개가 로우패스 필터의 특성을 보이고,특정 주파수에서 진폭이 커지는 공진현상을 발견할 수 있었으므로, 날개를 설계할때는 공명주파수를 너무 저주파는 아닌, 적당한 주파수 대역폭 사이에 잡아야 함을 알 수 있었다. 해당 탐구에서는 보다 더 확실한 날개의 로우패스 필터의 특성을 파악하기 위해, 여러개의 동일한 진폭을 가진 임의의 난기류 사인파를 만들어서, 그것에 대한 주파수 반응을 나타내보았다.

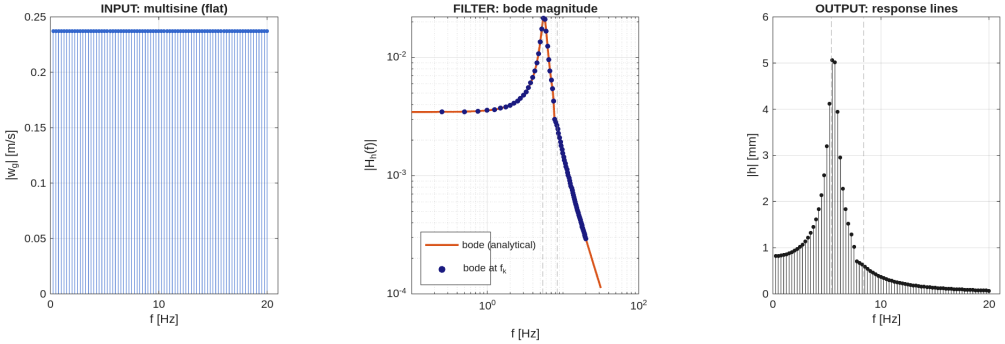
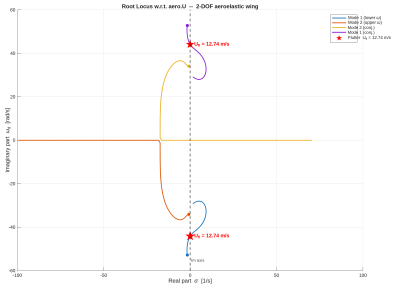


Figure 4: Frequency response of sine waves with rms=1.5

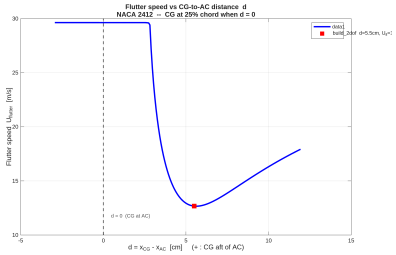
[3]von-karman model:$\Phi_w(\omega) = \frac{\sigma_w^2 L_w}{\pi U} \frac{1+3\,(L_w\,\omega/U)^2}{\left[1+(L_w\,\omega/U)^2\right]^2}$ where $\sigma_w = 1.5, L_w = 1.0, a = 1.339$

2.4 속도에 따른 고유값의 변화(WEEK5)

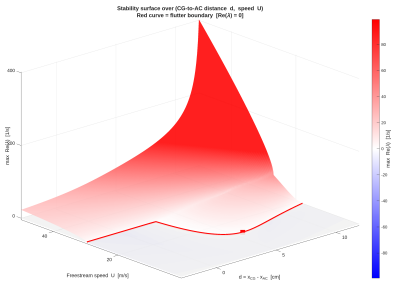
(2)에서, 공기력 행렬의 C_a, F_g는 U에 대한 함수임을 알 수 있다. 즉, 속도가 변함에 따라 system matrix **A**가 변하므로, 평균 순항속도U에 따라서 시스템의 λ[4]가 달라진다. 해당 시스템의 두개의 모드는 고유값의 허수부분에,감쇠는 실수에 연결되어져 있는것을 이용하기 root-locus plot를 사용하여 RHS를 넘어가는 고유값이 있나 살펴보았다.[5] 보이는



(a) Root-Locus plot w.r.t. velocity U. ★ indicate the boundary of stable.



(b) 공력중심과 무게중심 사이 간격 e_a에 따른 속도와 flutter boundary사이의 관계



(c) 공력중심과 무게중심 사이 간격 e_a에 따른 속도와 고유값 사이의 관계

Figure 5: 5주차 결과. 빨간색 네모는 시스템이 table 1일 때 시스템이 위치하는 곳이다.

그림과 같이, 위의 값들은 날개의 진동분석을 위해 비교적 불안정한 위치에 설계된 날개 모델임을 알 수 있다. 이러한 불안정성은 공력중심과 무게중심 사이의 간격을 일정 수준 줄임으로써fig. 5b해소할 수 있음을 볼 수 있었다. 그리고 이렇게 d를 바꾸었을 때에는, root locus의 모양또한 달라질 것이므로, 다시 해당 그래프를 그려서 flutter boundary 근처에서의 속도에 따른 고유값(λ)를 재조사하여 보다 안정적인 설계를 해야함 또한 알 수 있었다. 또한 위 경우에는 주파수 병합이 일어나는[6], 동일한 속도에 대해서 두 고윳값의 허수부분이 동일한 현상이 일어나지 않았지만, 다른 U에 대해서는 해당 경우가 발생할 수 있으므로, 이를 피해서 설계를 해야함을 알 수 있었다.

2.5 선형 비선형 모델의 비교(WEEK4)

위에서는 선형적인 경우만을 다루었으나, week4에서는 용수철이 비선형적으로 거동할 경우에 대해 탐구하였다. 비선형 시스템에서는 위와 같이 행렬 형태의 state space로 표현하지 못하여 bode,root locus와 같은 거동의 해석이 선형 시스템만큼 쉽지 않다. 해서 해당 탐구에서는 간단하게 impulse[7],multisine 함수를 수치해석기법(ode45)를 사용하여 선형과 비선형 거동의 차이를 아래와 같이 보였다.

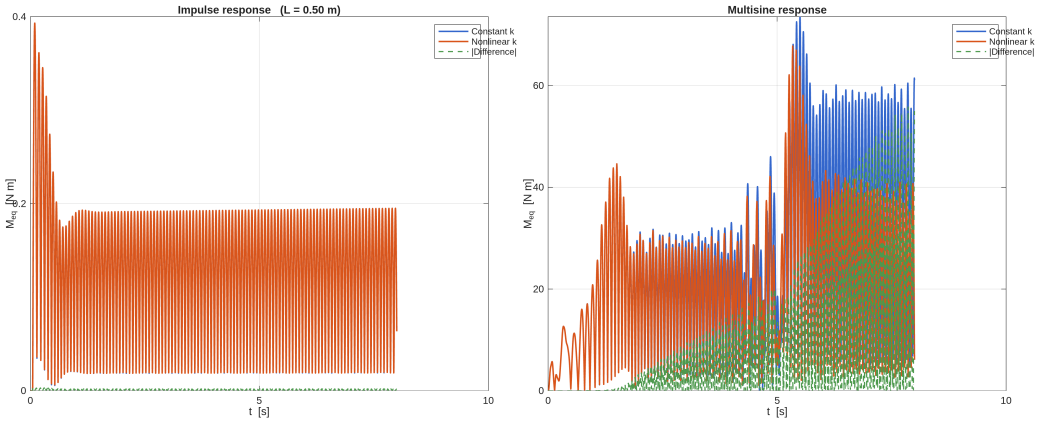


Figure 6: Impulse and multisine response of linear and nonlinear spring stiffness. velocity U is taken to flutter boundary speed to emphasize its different behavior, and cubic term used here is 900. Others are same as above.

보이는 바와 같이, 초기단계에서는 두개가 별 차이가 없으나, 시간이 갈수록 그 차이가 점점 커짐을 알 수 있다. 또한 선형에 비해 비선형에서 모멘트가 커지므로, 비선형의 경우에서 피로 또는 항복응력 초과의 확률이 더 높음을 알 수 있다.하지만, 주기적으로 힘이 작용하는 sine함수와 달리 impulse와 같은, 단기적으로 힘이 작용하는 경우에는, 선형, 비선형 스프링의 차이가 별로 없다는 점을 미뤄보아, 항공기의 속도가 빠르게 유지되어, 난기류 같은 순간적인 impulse로 작용하는 힘만 존재한다고 가정하면, 충분히 비선형 시스템은 선형 시스템으로 취급되어 해석을 할 수 있음을 알 수 있었다.

[4]eigenvalue
[5]켤레를 제외한, 두개의 고유한 고유값들 중 하나라도 root-locus plot 에서 RHS로 넘어가면 시스템은 발산한다
[6]두 모드가 동일한 고유진동수를 가지어,동시에 공진현상이 발생해 큰 피해를 주는 현상
[7]FAA 14 CFR §25.341(a)

3 결론: 최종 설계안

5주에 걸친 해석을 종합하면, 안정적이면서 경량인 날개는 아래와 같은 최소 세가지 시선을 만족해야함을 알 수 있다.
1,2주차 질량&강성 trade-off에서 공진 진폭을 피로 한계 이내로 누르는 최소 강성 $k \approx 5500\,\mathrm{N\,m^{-1}}$

3주차 주파수 응답 공진 대역폭을 난류 저주파 공진주파수 밖으로 밀어내는 w_d

5주차 플러터 한계를 최고 비행속도 $V_{\max}$ 이상으로 확보하기 위해 무게중심–탄성중심 간격 e_a
4주차에서 보였듯,지속적으로 몰아치는 드문 난기류의 경우를 제외한,임펄스,혹은 시간이 짧은 외부충격의 경우 비선형 효과가 충분히 작아,선형효과만으로 설계 검증이 타당함을 위에서 보였다.비선형 조건을 제외하고,선형시스템에서 위 세가지 조건을 모두 만족하는 물리량들은 table 2에 정리하였다.나오지 않은 물리량들은 table 1의 값들이다.

Table 2: 종합 해석으로부터 도출한 권장 설계 파라미터.

설계값	근거 (주차)	권장 방향 / 값
간격 e_a	플러터 한계 (5주)	$d \le 2.52\,\mathrm{cm}$ ($V_f = 29.6\,\mathrm{m/s} > V_{\max}$)
공진주파수[8] w_d	로우패스 대역 (3주)	$w_d \ge 13.6\,\mathrm{Hz}$
강성[9] k	질량–강성 trade-off ($1 \cdot 2$주)	$k \gtrsim 5500\,\mathrm{N\,m^{-1}}$
비선형 항	돌풍 응답 (4주)	순항조건에서 무시 가능

A Structural Model from Energy

A.1 Mass Matrix — Kinetic Energy

Let the section have mass m, centre of gravity a distance x_{cg}^{p} aft of P, and moment of inertia I_{cg} about the c.g. From (1), the downward velocity of the c.g. is $\dot{z}_{\mathrm{cg}} = \dot{h} + x_{\mathrm{cg}}^{p}\dot{\theta}$, so the kinetic energy is

$$T = \tfrac{1}{2} m\,(\dot{h} + x_{\mathrm{cg}}^{p}\dot{\theta})^2 + \tfrac{1}{2} I_{\mathrm{cg}}\dot{\theta}^2 = \tfrac{1}{2} m\dot{h}^2 + S\,\dot{h}\dot{\theta} + \tfrac{1}{2} I_p\,\dot{\theta}^2, \tag{4}$$

where the **static unbalance** and **inertia about** P are defined as

$$S \equiv m\,x_{\mathrm{cg}}^{p}, \qquad I_p \equiv I_{\mathrm{cg}} + m\,(x_{\mathrm{cg}}^{p})^2. \tag{5}$$

Writing $T = \tfrac{1}{2}\dot{\mathbf{q}}^{\mathsf{T}}\mathbf{M}\dot{\mathbf{q}}$ with $\mathbf{q} = [\,h,\ \theta\,]^{\mathsf{T}}$ gives

$$\boxed{\mathbf{M} = \begin{bmatrix} m & S \\ S & I_p \end{bmatrix}} \tag{6}$$

A.2 Stiffness Matrix — Spring Potential Energy

The two springs sit at $\xi = -d/2$ (front) and $\xi = +d/2$ (rear). Their deflections are $z_1 = h - \frac{d}{2}\theta$ and $z_2 = h + \frac{d}{2}\theta$, so

$$V = \tfrac{1}{2} k z_1^2 + \tfrac{1}{2} k z_2^2 = \tfrac{1}{2} k\left[\left(h - \tfrac{d}{2}\theta\right)^2 + \left(h + \tfrac{d}{2}\theta\right)^2\right] = k\,h^2 + \tfrac{1}{4} k d^2\,\theta^2. \tag{7}$$

The cross terms $\mp d\,h\theta$ cancel because P is the midpoint. With $V = \tfrac{1}{2}\mathbf{q}^{\mathsf{T}}\mathbf{K}_s\mathbf{q}$,

$$\boxed{\mathbf{K}_s = \begin{bmatrix} 2k & 0 \\ 0 & \tfrac{1}{2} k d^2 \end{bmatrix}} \tag{8}$$

A.3 Damping Matrix — Rayleigh Dissipation

The dampers act in parallel with the springs, so the Rayleigh dissipation function is identical in form to (7) with $k \to c$ and $z \to \dot{z}$: $\mathcal{D} = \tfrac{1}{2} c\dot{z}_1^2 + \tfrac{1}{2} c\dot{z}_2^2$, giving[10]

$$\boxed{\mathbf{C} = \begin{bmatrix} 2c & 0 \\ 0 & \tfrac{1}{2} c d^2 \end{bmatrix}} \tag{9}$$

[9] 위에 서술한 Von Karman식에 V_f값 대입시 값 감소가 뚜렷해지는 지점을 $w_{d,\mathrm{crit}}$로 지정하였다

[10] For a viscous damper $F_v = c\dot{z}$, the Rayleigh dissipation function is $\mathcal{D} = \tfrac{1}{2} c\dot{z}^2$, which represents half the rate of energy dissipation (power) of the system.

A.4 Structural Equations of Motion

Lagrange's equations, $\frac{d}{dt}(\partial T/\partial \dot{q}_i) - \partial T/\partial q_i + \partial V/\partial q_i + \partial \mathcal{D}/\partial \dot{q}_i = Q_i$, yield

$$\mathbf{M}\ddot{\mathbf{q}} + \mathbf{C}\dot{\mathbf{q}} + \mathbf{K}_s\mathbf{q} = \mathbf{Q}_{\text{aero}}, \tag{10}$$

where $\mathbf{Q}_{\text{aero}}$ denotes the generalised aerodynamic forces derived next.

A.5 Aerodynamic Forces

The lift acts at the aerodynamic centre, a distance e_a *forward* of P (i.e., $\xi_{\text{ac}} = -e_a$). A vertical gust of velocity w_g adds an incremental angle of attack w_g/U to the geometric pitch θ, where U is the flight speed. Furthermore, when the wing plunges downward at velocity $\dot{h}$, it experiences an upward relative wind, creating an induced angle of attack of $\frac{\dot{h}}{U}$. Under the quasi-steady approximation, lift is proportional to the effective angle of attack through $C_{l\alpha}$:

$$L = \bar{q}S\,C_{l\alpha}\Big(\theta + \frac{\dot{h}}{U} + \frac{w_g}{U}\Big), \qquad \bar{q}S = \tfrac{1}{2}\rho U^2 c\,b \tag{11}$$

For a NACA 2412 airfoil, $C_{l\alpha} = \text{Const}(\approx 2\pi)$, and $\bar{q}S$ is the dynamic pressure multiplied by the planform area.[11] The virtual work of the upward lift under a virtual displacement of P gives the generalised forces:

$$Q_h = -L, \qquad Q_\theta = +L\,e_a, \tag{12}$$

The constant aerodynamic-centre moment M_{ac} only shifts the trim and is dropped from the dynamics. Splitting (11) into a part driven by the motion (θ, $\dot{h}$, $\dot{\theta}$) and a part driven by the gust w_g,

$$\begin{bmatrix} Q_h \\ Q_\theta \end{bmatrix} = \underbrace{\bar{q}S\,C_{l\alpha}\begin{bmatrix} 0 & -1 \\ 0 & e_a \end{bmatrix}\begin{bmatrix} h \\ \theta \end{bmatrix} + \frac{\bar{q}S\,C_{l\alpha}}{U}\begin{bmatrix} -1 & 0 \\ e_a & 0 \end{bmatrix}\begin{bmatrix} \dot{h} \\ \dot{\theta} \end{bmatrix}}_{\text{motion-dependent}} + \underbrace{\frac{\bar{q}S\,C_{l\alpha}}{U}\begin{bmatrix} -1 \\ e_a \end{bmatrix} w_g}_{\text{gust forcing}}. \tag{13}$$

Moving the motion-dependent part to the left of (10) defines the **aerodynamic stiffness** $\mathbf{K}_a$, **aerodynamic damping** $\mathbf{C}_a$, and **gust force vector** $\mathbf{F}_g$:

$$\boxed{\mathbf{K}_a = \bar{q}S\,C_{l\alpha}\begin{bmatrix} 0 & 1 \\ 0 & -e_a \end{bmatrix}, \qquad \mathbf{C}_a = \frac{\bar{q}S\,C_{l\alpha}}{U}\begin{bmatrix} 1 & 0 \\ -e_a & 0 \end{bmatrix}, \qquad \mathbf{F}_g = \frac{\bar{q}S\,C_{l\alpha}}{U}\begin{bmatrix} -1 \\ e_a \end{bmatrix}.} \tag{14}$$

Note that $\mathbf{K}_a$ is *not* symmetric: aerodynamic loads are non-conservative follower forces. This asymmetry is harmless at low speed but is the very mechanism that produces flutter at high speed.

A.6 Assembled System

Collecting the structural matrices (6), (8), (9) and the aerodynamic matrices derived above, and moving the motion-dependent aerodynamic terms to the left-hand side, reproduces the governing equation stated in the main text, eq. (2), with $\mathbf{K}_{\text{eff}} = \mathbf{K}_s + \mathbf{K}_a$ and $\mathbf{D}_{\text{tot}} = \mathbf{C} + \mathbf{C}_a$.

A.7 State-Space Conversion

With the state vector $\mathbf{x} = [\mathbf{q},\ \dot{\mathbf{q}}]^\top$, solving eq. (2) for $\ddot{\mathbf{q}}$,

$$\ddot{\mathbf{q}} = -\mathbf{M}^{-1}\mathbf{K}_{\text{eff}}\,\mathbf{q} - \mathbf{M}^{-1}\mathbf{D}_{\text{tot}}\,\dot{\mathbf{q}} + \mathbf{M}^{-1}\mathbf{F}_g\,w_g, \tag{15}$$

and stacking this with the trivial identity $\dot{\mathbf{q}} = \dot{\mathbf{q}}$ yields the first-order form $\dot{\mathbf{x}} = \mathbf{A}\mathbf{x} + \mathbf{B}w_g$, $\mathbf{y} = \mathbf{C}_{\text{out}}\mathbf{x}$ with

$$\mathbf{A} = \begin{bmatrix} \mathbf{0}_{2\times 2} & \mathbf{I}_2 \\ -\mathbf{M}^{-1}\mathbf{K}_{\text{eff}} & -\mathbf{M}^{-1}\mathbf{D}_{\text{tot}} \end{bmatrix}, \quad \mathbf{B} = \begin{bmatrix} \mathbf{0}_{2\times 1} \\ \mathbf{M}^{-1}\mathbf{F}_g \end{bmatrix}, \quad \mathbf{C}_{\text{out}} = \begin{bmatrix} \mathbf{I}_2 & \mathbf{0}_{2\times 2} \end{bmatrix}, \quad \mathbf{D}_{\text{out}} = \mathbf{0}_{2\times 1}. \tag{16}$$

The output matrix $\mathbf{C}_{\text{out}}$ selects the displacements $\mathbf{y} = [h,\ \theta]^\top$; other outputs (velocities, accelerations, or the root bending moment) are obtained by modifying $\mathbf{C}_{\text{out}}$ and $\mathbf{D}_{\text{out}}$ accordingly.

[11] The unsteady Theodorsen/Sears corrections are omitted. This slightly overestimates the peaks but is appropriate for the present reduced-order model.

B 소스코드

B.1 저장소 구조

전체 소스코드는 아래 GitHub 저장소에 공개되어 있다.

`https://github.com/sekyoung/2026-1_vibration_project`

주차별 분석 스크립트는 `srcs/` 아래 주차 폴더에, 공통 유틸리티는 `srcs/` 루트에 위치한다.

```
srcs/
  ├── build_models.m              % 선형/비선형 상태공간 행렬 생성 및 .mat 저장
  ├── modal_analysis.m            % 고유값·감쇠비 분석 및 극점 지도 출력
  ├── week1.jl                    % 1주차: curve-fitting 등가모델 (Julia)
  ├── week2.jl                    % 2주차: 가진 응답 분석 (Julia)
  ├── week2/
  │     ├── fourier_torque.jl      % 로터리 엔진 토크의 푸리에 분해
  │     └── tangential_force.jl    % 접선력 산출
  ├── week3/
  │     ├── bode_response.m        % Bode 선도
  │     ├── impulse_response.m     % 임펄스 응답
  │     ├── multisine_analysis.m   % 멀티사인 주파수 응답 분석
  │     ├── multisine_function_generator.m
  │     └── multisine_toolbox_response.m
  ├── week4/
  │     ├── impulse_all.m          % 선형·비선형 임펄스 비교
  │     ├── impulse_function_generator.m
  │     ├── moment_response.m      % 굽힘 모멘트 응답
  │     ├── multisine_function_generator.m
  │     ├── nonlinear_response.m   % 비선형 적분 (ode45)
  │     └── result_plot.m
  └── week5/
        ├── flutter_vs_d.m         % 스프링 간격 d에 따른 플러터 경계
        └── root_locus_U.m         % 속도에 따른 root-locus
```

B.2 공통 유틸리티 코드

B.2.1 build_models.m

모든 주차 스크립트가 공유하는 `.mat` 데이터 파일을 생성하는 진입점이다. 내부적으로 두 함수를 호출한다: `build_2dof()`는 선형 2DOF 모델(`wing_2dof.mat`)을, `build_nl()`은 각 스프링에 3차 비선형 항을 포함한 모델(`wing_nl.mat`)을 조립하여 저장한다. 공통 공력 행렬($\mathbf{K}_a, \mathbf{C}_a, \mathbf{F}_g$)은 `aero_terms()`에서 한 번만 계산한 뒤 두 모델에 재사용된다.

```
function build_models()
% build_models  Builds both Week-4 model files in this directory:
%   wing_2dof.mat -- linear 2-DOF model (Week-3 parameters, identical output)
%   wing_nl.mat   -- per-spring model with cubic stiffness + q=0 linearisation
%
% Replaces the need for week3/build_ss.m and week4/build_ss_nl.m at runtime.
%{
CF: Difference between build_models.m and the original build_ss.m and build_ss_nl.m:
build_models.m has two functions, build_2dof() and build_nl(), that construct the same physical models as
     build_ss.m and build_ss_nl.m, respectively. The parameters
Comparing the two files, build_models.m's build_nl() function covers all the same physics and exports
     identical variables to wing_nl.mat. But there is one missing piece:

Missing from build_models.m: the diagnostic fprintf statements that build_ss_nl.m prints (lines 73-75):

The full Ks matrix display (disp(Ks))
The coupling term report: Coupling term K_h,theta = ... (0 when k1 = k2)
build_models.m only prints the natural frequencies line, while build_ss_nl.m also prints the structural
     stiffness matrix and the off-diagonal coupling term.

Everything else -- parameters, matrix assembly, state-space construction, saved variables -- is identical. If
     those diagnostic prints matter to you, they can be added to build_nl().
%}
here = fileparts(mfilename('fullpath'));
build_2dof(here);
build_nl(here);
end

% ========================================================================

function [qS, Ka, Ca, Fgust, aero] = aero_terms()
aero.rho     = 1.225;    % air density           [kg/m^3]
aero.U       = 12.74;     % freestream speed      [m/s]
aero.chord   = 0.30;     % airfoil chord         [m]
aero.span    = 0.50;     % section span          [m]
```

```
aero.Clalpha = 6.0;    % lift-curve slope      [1/rad]
aero.ea      = 0.045;    % aero centre fwd of reference [m]

qS    = 0.5 * aero.rho * aero.U^2 * aero.chord * aero.span;
Ka    = [0,  qS*aero.Clalpha;
         0, -qS*aero.Clalpha*aero.ea];
Ca    = (qS*aero.Clalpha/aero.U) .* [1,        0;
                                     -aero.ea, 0];
Fgust = (qS*aero.Clalpha/aero.U) .* [-1; aero.ea];
end

% ------------------------------------------------------------------------

function build_2dof(here)
% Week-3 linear model, lumped springs (k each side, symmetric).
m  = 1.5;       % section mass                [kg]
Ip = 0.0084;      % pitch inertia               [kg*m^2]
S  = 0.0225;     % static unbalance            [kg*m]
k  = 5500;    % each spring stiffness        [N/m]
cd = 1.0;       % each damper coefficient      [N*s/m]
d  = 0.22;      % distance between springs     [m]

[~, Ka, Ca, Fgust, aero] = aero_terms();

M_mat = [m, S;
         S, Ip];
C_mat = [2*cd, 0;
         0,    cd*d^2/2];
Ks    = [2*k, 0;
         0,   k*d^2/2];

Keff   = Ks + Ka;
Ctotal = C_mat + Ca;

[A, B, C_out, D_out] = to_ss(M_mat, Keff, Ctotal, Fgust);
fn_struct = sqrt(eig(M_mat \ Ks)) / (2*pi);

U = aero.U;  rho = aero.rho;  chord = aero.chord;
span = aero.span;  Clalpha = aero.Clalpha;  ea = aero.ea;

save(fullfile(here, 'wing_2dof.mat'), ...
     'A','B','C_out','D_out', ...
     'M_mat','Keff','Ctotal','Fgust', ...
     'U','rho','chord','span','Clalpha','ea', ...
     'fn_struct');
fprintf('wing_2dof.mat  fn = %.3f / %.3f Hz\n', fn_struct(1), fn_struct(2));
end

% ------------------------------------------------------------------------

function build_nl(here)
% Per-spring model with cubic stiffness (cubic lives in the solver only).
m  = 1.5;       % section mass                [kg]
Ip = 0.0084;      % pitch inertia               [kg*m^2]
S  = 0.0225;     % static unbalance            [kg*m]
d  = 0.22;      % distance between springs     [m]

k1   = 900.0;  % spring 1 stiffness           [N/m]
k2   = 900.0;  % spring 2 stiffness           [N/m]   !(set != k1 for h-theta coupling)
knl1 = 1000;     % spring 1 cubic coefficient   [N/m^3] !(set > 0 for hardening)
knl2 = 1000;     % spring 2 cubic coefficient   [N/m^3]
c1   = 1;     % damper 1 coefficient         [N*s/m]
c2   = 1;     % damper 2 coefficient         [N*s/m]

r1 = +d/2;      % spring/damper 1 moment arm   [m]
r2 = -d/2;      % spring/damper 2 moment arm   [m]

[~, Ka, Ca, Fgust, aero] = aero_terms();

M_mat = [m, S;
         S, Ip];
Ks    = [k1    + k2,     k1*r1   + k2*r2;
         k1*r1 + k2*r2,  k1*r1^2 + k2*r2^2];
C_struct = [c1    + c2,     c1*r1   + c2*r2;
            c1*r1 + c2*r2,  c1*r1^2 + c2*r2^2];
C_mat = C_struct + Ca;

[A, B, C_out, D_out] = to_ss(M_mat, Ks + Ka, C_mat, Fgust);
fn_struct = sqrt(eig(M_mat \ Ks)) / (2*pi);
U = aero.U;

save(fullfile(here, 'wing_nl.mat'), ...
     'A','B','C_out','D_out','fn_struct', ...
     'M_mat','C_mat','Ka','Fgust', ...
     'r1','r2','k1','k2','knl1','knl2', ...
     'U');
fprintf('wing_nl.mat    fn = %.3f / %.3f Hz\n', fn_struct(1), fn_struct(2));
end

% ------------------------------------------------------------------------

function [A, B, C_out, D_out] = to_ss(M, K, C, Fgust)
```

```
Minv  = inv(M);
A     = [zeros(2),  eye(2);
         -Minv*K,  -Minv*C];
B     = [zeros(2,1);
         Minv*Fgust];
C_out = [eye(2), zeros(2)];
D_out = zeros(2,1);
end
```

Listing 1: `build_models.m` — 상태공간 모델 생성

B.2.2 `modal_analysis.m`

`wing_2dof.mat`을 불러와 구조 고유값($\mathbf{A}_{\text{struct}}$)과 공력탄성 고유값($\mathbf{A}$)을 각각 계산한다. 내부 함수 `classify_eigs()`가 켤레 복소 극점(부족감쇠)과 실수 극점(과감쇠 · 발산)을 자동으로 분류하여 고유진동수 f_n, 감쇠비 ζ를 표로 출력하고 극점 지도를 그린다.

```
% modal_analysis.m  --  Damping ratios and natural frequencies from wing_2dof.mat
%
% Two sets of results are computed:
%   1. Structural modes  : eigenvalues of  M_mat \ Ks  (no aerodynamics)
%   2. Aeroelastic modes : eigenvalues of  A           (aero included)
%
% Eigenvalue types:
%   Complex pair  ->  wn = |lambda|,  zeta = -Re(lambda)/wn,  fn = wn/(2*pi)
%   Real negative ->  overdamped / stable pole,   reported as |lambda| [rad/s]
%   Real positive ->  divergence / unstable pole, reported as |lambda| [rad/s]

here = fileparts(mfilename('fullpath'));
load(fullfile(here, 'wing_2dof.mat'), ...
     'A', 'M_mat', 'Keff', 'Ctotal', ...
     'U', 'rho', 'chord', 'span', 'Clalpha', 'ea');

%% -- Reconstruct structural matrices ----------------------------------------
qS = 0.5 * rho * U^2 * chord * span;
Ka = [0,  qS*Clalpha;
      0, -qS*Clalpha*ea];
Ca = (qS*Clalpha/U) .* [1,   0;
                        -ea, 0];

Ks       = Keff - Ka;
C_struct = Ctotal - Ca;
Minv     = inv(M_mat);

%% -- Build structural A-matrix and get all eigenvalues ----------------------
A_struct = [zeros(2), eye(2);
            -Minv*Ks, -Minv*C_struct];

lam_s  = eig(A_struct);
lam_ae = eig(A);

%% -- Helper: classify and extract modal parameters --------------------------
[s_wn,  s_zeta,  s_fn,  s_type,  lam_s_plot]  = classify_eigs(lam_s);
[ae_wn, ae_zeta, ae_fn, ae_type, lam_ae_plot] = classify_eigs(lam_ae);

%% -- Print structural results -----------------------------------------------
fprintf('\n=== Structural Modes (no aerodynamics) ===\n');
fprintf('  %-6s  %-14s  %-14s  %-12s  %s\n', ...
        'Mode', 'fn [Hz]', 'wn [rad/s]', 'zeta [-]', 'Type');
for i = 1:numel(s_wn)
    if isnan(s_zeta(i))
        fprintf('  %-6d  %14.4f  %14.4f  %12s  %s\n', ...
            i, s_fn(i), s_wn(i), 'N/A', s_type{i});
    else
        fprintf('  %-6d  %14.4f  %14.4f  %12.6f  %s\n', ...
            i, s_fn(i), s_wn(i), s_zeta(i), s_type{i});
    end
end

%% -- Print aeroelastic results ----------------------------------------------
fprintf('\n=== Aeroelastic Modes (U = %.1f m/s) ===\n', U);
fprintf('  %-6s  %-14s  %-14s  %-12s  %s\n', ...
        'Mode', 'fn [Hz]', 'wn [rad/s]', 'zeta [-]', 'Type');
for i = 1:numel(ae_wn)
    if isnan(ae_zeta(i))
        fprintf('  %-6d  %14.4f  %14.4f  %12s  %s\n', ...
            i, ae_fn(i), ae_wn(i), 'N/A', ae_type{i});
    else
        fprintf('  %-6d  %14.4f  %14.4f  %12.6f  %s\n', ...
            i, ae_fn(i), ae_wn(i), ae_zeta(i), ae_type{i});
    end
end
fprintf('\n');

%% -- Store results ----------------------------------------------------------
results.structural.fn_Hz   = s_fn;
results.structural.wn_rads = s_wn;
results.structural.zeta    = s_zeta;
```

```
results.structural.type    = s_type;

results.aeroelastic.fn_Hz   = ae_fn;
results.aeroelastic.wn_rads = ae_wn;
results.aeroelastic.zeta    = ae_zeta;
results.aeroelastic.type    = ae_type;

%% -- Pole map ---------------------------------------------------------
figure('Name','Pole Map','Position',[100 100 620 460]);
hold on; grid on;

% structural: plot both upper and mirrored lower half
s_conj  = [lam_s_plot;  conj(lam_s_plot(imag(lam_s_plot) ~= 0))];
ae_conj = [lam_ae_plot; conj(lam_ae_plot(imag(lam_ae_plot) ~= 0))];

scatter(real(s_conj),  imag(s_conj),  70, 'r', '^', 'LineWidth', 1.5, ...
        'DisplayName', 'Structural');
scatter(real(ae_conj), imag(ae_conj), 70, 'b', 'o', 'filled', ...
        'DisplayName', 'Aeroelastic');

xline(0, 'k--', 'LineWidth', 0.8, 'HandleVisibility', 'off');
yline(0, 'k:',  'LineWidth', 0.5, 'HandleVisibility', 'off');
xlabel('Real  [1/s]');
ylabel('Imag  [rad/s]');
title(sprintf('Pole Map  (U = %.1f m/s)', U));
legend('Location', 'best');

%% -- Local function ---------------------------------------------------
function [wn, zeta, fn, type, lam_keep] = classify_eigs(lam)
% Separates complex (underdamped) and real (overdamped/unstable) eigenvalues.
% Returns one entry per unique mode.
    tol = max(abs(lam)) * 1e-8;

    is_complex = abs(imag(lam)) > tol;
    lam_c = lam(is_complex & imag(lam) > 0);   % upper half-plane of conjugate pairs
    lam_r = real(lam(~is_complex));             % real poles

    % sort: underdamped by ascending frequency, real by ascending value
    [~, idx] = sort(abs(lam_c));
    lam_c = lam_c(idx);
    lam_r = sort(lam_r);

    wn_c   = abs(lam_c);
    zeta_c = -real(lam_c) ./ wn_c;
    fn_c   = wn_c / (2*pi);
    type_c = cell(numel(lam_c), 1);
    for k = 1:numel(lam_c)
        if zeta_c(k) >= 0
            type_c{k} = 'underdamped (stable)';
        else
            type_c{k} = 'underdamped (unstable/flutter)';
        end
    end

    wn_r   = abs(lam_r);
    fn_r   = wn_r / (2*pi);
    zeta_r = nan(numel(lam_r), 1);
    type_r = cell(numel(lam_r), 1);
    for k = 1:numel(lam_r)
        if lam_r(k) < 0
            type_r{k} = 'real (overdamped/stable)';
        else
            type_r{k} = 'real (divergent/unstable)';
        end
    end

    wn   = [wn_c(:);   wn_r(:)];
    zeta = [zeta_c(:); zeta_r(:)];
    fn   = [fn_c(:);   fn_r(:)];
    type = [type_c;    type_r];
    lam_keep = [lam_c(:); lam_r(:)];
end
```

Listing 2: `modal_analysis.m` — 고유값 분석 및 극점 지도

B.3 주차별 Pluto 노트북 (Julia)(WEEK1,WEEK2)

Pluto.jl은 Julia 기반의 반응형 노트북 환경으로, 각 셀은 파일 내에서 `# ╔═╡ UUID` 주석으로 구분된다. 아래 코드에서 해당 주석이 회색으로 표시되어 셀 경계 역할을 한다. 패키지 의존성 잠금 파일(`PLUTO_*_TOML_CONTENTS`)은 자동 생성 코드이므로 제외하였다. julia를 다운받아서 pluto를 통해 여는것이 제일 추천되는 방식이다.

B.3.1 `week1.jl` — 1주차: curve-fitting 등가모델

`CSV.jl`로 Ground Vibration Test 데이터를 불러온 뒤 Butterworth 대역통과 필터를 적용하고, Gradient Descent / Gauss-Newton / CurveFit 세 알고리즘으로 자유진동 해 $x(t) = Xe^{-\zeta\omega_n t}\cos(\omega_d t - \phi)$의 파라미터를 피팅한다. 마지막 셀에서는 감쇠비 ζ를 sweep하여 임계감쇠 근방 거동의 차이를 시각화한다.

week1.jl

```
### A Pluto.jl notebook ###
# v0.20.25

using Markdown
using InteractiveUtils

#  44099108-4887-45f5-818c-8c1e258249f0
using CSV, DataFrames, Plots, FFTW, DSP, CurveFit, ForwardDiff, Statistics, LinearAlgebra,LaTeXStrings,PlutoUI

#  f3d44c6a-0090-43a2-93e1-f2cdd16bb6cd
md"""
# WEEK 1
"""

#  2c6c9ec3-2c74-4dd6-aea3-5481301d9185
df = CSV.read("./resources/week1/week1_wing_gvt_data.csv",DataFrame);

#  b1041ed1-092f-4ecb-832a-95389f058ba1
begin
        t = df[!, "Time (s)"]
        x = df[!, "Displacement (m)"]

        plot(t, x,
            xlabel = "Time (s)",
            ylabel = "Displacement (m)",
            title  = "Week 1 Wing GVT - Free Vibration",
            legend = false,
            lw     = 1.5,
            color  = :steelblue
        )

end

#  d9433d2e-1be4-4846-917d-164cc5fb88b6
begin
        N  = length(t)
        dt = t[2] - t[1]
        fs = 1.0 / dt                   # sampling frequency (Hz)
        # --- Step 1: FFT spectrum (before filter) ---
        X     = abs.(fft(x))[1:N÷2]
        freqs = (0:N÷2-1) .* (fs / N)

        dominant_freq = freqs[argmax(X)]
                # --- Step 2: Butterworth band-pass filter ---
margin = 2.0
f_low  = max(0.1, dominant_freq - margin)
f_high = dominant_freq + margin

nyquist    = fs / 2.0
bpf        = digitalfilter(Bandpass(f_low / nyquist, f_high / nyquist), Butterworth(4))
x_filtered = filtfilt(bpf, x)
                # --- Step 3: Spectrogram ---
        win_size = 128
        overlap  = win_size ÷ 2
        sg       = spectrogram(x, win_size, overlap; fs=fs)
                "Dominant frequency: $(round(dominant_freq, digits=3)) Hz (ωd = $(round(dominant_freq*2π,digits=3)) rad/s)"

end

#  b4bd090d-e66a-4b00-8803-aee9e40a0267
begin
        # --- Plot 1: Raw vs Filtered ---
        p1_ = plot(t, x,
            label  = "Raw",
            xlabel = "Time (s)", ylabel = "Displacement (m)",
            title  = "GVT Signal - Raw vs Filtered",
            lw = 1.5, alpha = 0.8, color = :gray)
        plot!(p1_, t, x_filtered,
            label = "Butterworth BP ($(round(f_low,digits=1))-$(round(f_high,digits=1)) Hz, order 4)",
            lw = 2.0, color = :steelblue)

                # --- FFT spectrum after filter ---
        X_filtered = abs.(fft(x_filtered))[1:N÷2]

        # --- Plot 2: FFT Spectrum before & after filter ---
        xlim_max = min(fs/2, dominant_freq * 5)
        p2_ = plot(freqs, X,
            xlabel = "Frequency (Hz)", ylabel = "Amplitude",
            title  = "FFT Spectrum - Before vs After Filter",
            legend = true, lw = 1.5, color = :darkorange,
            xlims  = (0, xlim_max),
            label  = "Before filter")
        plot!(p2_, freqs, X_filtered,
            lw = 1.5, color = :steelblue, alpha = 0.8,
            label = "After filter")
        vline!(p2_, [dominant_freq],
            linestyle = :dash, color = :red, lw = 1.5,
            label = "Dominant: $(round(dominant_freq, digits=2)) Hz")
        vline!(p2_, [f_low, f_high],
            linestyle = :dot, color = :gray, lw = 1.0,
            label = "Band-pass range")
```

```
            # --- Plot 3: Spectrogram ---
        p3_ = heatmap(sg.time, sg.freq, pow2db.(sg.power),
            xlabel  = "Time (s)", ylabel = "Frequency (Hz)",
            title   = "Spectrogram",
            color   = :inferno,
            ylims   = (0, min(fs/2, dominant_freq * 5)),
            clims   = (maximum(pow2db.(sg.power)) - 60, maximum(pow2db.(sg.power))))
        hline!(p3_, [dominant_freq],
            linestyle = :dash, color = :cyan, lw = 1.5,
            label = "Dominant freq")

        p_ = plot(p1_, p2_, p3_, layout = (3, 1), size = (1000, 1000))

end

#  98d8c9f2-36bf-4e8b-af3c-d4c7b729c4ea
md"""
 If we neglect bending and torsional coupling,such caused by structural couple,sweep effects etc. then approximate wing as simple as simple beam, moving along with vertical direction. from elastic theory, its well known
`` \delta = \frac{Wl^3}{3EI}``
where ``I = I_{zz} = \frac{bh^3}{12}`` . a typical airfoil, assume *NACA2412* in this case, has maximum 0.12 ratio of horizontal and vertical length. letting ``h=0.12b`` and ``k = \frac{3EI}{l^3}``.

Therefore, its reducing to first order free vibration analysis.

``m\ddot{x} + c\dot{x} + kx = \sum_{i=1}^{\infty} H(iw) e^{iwt} = 0``

the solution $x(t)$, is given as plot above.
"""

#  9bcbce35-937b-494b-9a59-76c5d9a63cbb
md"""
by conventional usage of constant in vibration theory, we assume these constant

``
ζ = \frac{c}{c_c}``

``
w_n = \sqrt{\frac{m}{k}}``


`` c_c = 2mw_n``

``w_d = w_n\sqrt{1-\zeta^2}``

The follwing graph is showing the damping is underdamped, therefore we can express the displacement ``x(t)`` in form of


``
x(t) = Xe^{-ζw_{n}t}\cos(\sqrt{1-\zeta^2}w_{n}t-)``

Using curve-fitting algorithms,
"""


#  7443122e-1607-430c-99d0-b7d5a2984c7f
begin
        t_arr=collect(t)
        x_arr = x_filtered
        f_dom = dominant_freq
# ─── Free-vibration model ──────────────────────────────────────────────────────
# p = [X_amp, ζ, ωn, ϕ]
function fv_model(p, t_vec)
    X_amp, zeta, wn, phi = p[1], p[2], p[3], p[4]
    wd = sqrt(max(1.0 - zeta^2, 1e-10)) * wn
    return @. X_amp * exp(-zeta * wn * t_vec) * cos(wd * t_vec - phi)
end

p0   = [maximum(abs.(x_arr)), 0.05, 2π * f_dom, 0.0]
loss(p) = sum((fv_model(p, t_arr) .- x_arr).^2)
mae(p)  = mean(abs.(fv_model(p, t_arr) .- x_arr))
mse(p)  = mean((fv_model(p, t_arr) .- x_arr).^2)

results    = Dict{String, Vector{Float64}}()
errors     = Dict{String, Float64}()
mse_errors = Dict{String, Float64}()

# ─── Algorithm 1: CurveFit.jl (NonlinearSolve auto-polyalgorithm) ──────────────
try
    prob = NonlinearCurveFitProblem(fv_model, p0, t_arr, x_arr)
    sol  = solve(prob)
    results["CurveFit (auto)"]    = sol.u
    errors["CurveFit (auto)"]     = mae(sol.u)
    mse_errors["CurveFit (auto)"] = mse(sol.u)
    println("CurveFit converged=$(isconverged(sol))  MAE=$(round(errors["CurveFit (auto)"],digits=7))")
catch e
    println("CurveFit failed: $e")
end

# ─── Algorithm 2: Gauss-Newton (ForwardDiff Jacobian + Tikhonov) ───────────────
function gauss_newton(model, t_vec, y, p_init; maxiter=500, tol=1e-10)
```

```
    p = copy(float.(p_init))
    for _ in 1:maxiter
        r  = model(p, t_vec) .- y
        J  = ForwardDiff.jacobian(pp -> model(pp, t_vec), p)
        dp = (J'J + 1e-8 * I) \ (J' * r)
        p .-= dp
        norm(dp) < tol && break
    end
    return p
end

try
    p_gn = gauss_newton(fv_model, t_arr, x_arr, p0)
    results["Gauss-Newton"]    = p_gn
    errors["Gauss-Newton"]     = mae(p_gn)
    mse_errors["Gauss-Newton"] = mse(p_gn)
    println("Gauss-Newton       MAE=$(round(errors["Gauss-Newton"],digits=7))")
catch e
    println("Gauss-Newton failed: $e")
end

# ── Algorithm 3: Gradient Descent with Armijo backtracking ──────────────────
function gradient_descent(loss_fn, p_init; maxiter=3000, tol=1e-8)
    p = copy(float.(p_init))
    for _ in 1:maxiter
        g  = ForwardDiff.gradient(loss_fn, p)
        norm(g) < tol && break
        lr = 1e-5
        L0 = loss_fn(p)
        for _ in 1:30
            loss_fn(p .- lr .* g) < L0 - 1e-4 * lr * dot(g, g) && break
            lr *= 0.5
        end
        p .-= lr .* g
    end
    return p
end

try
    p_gd = gradient_descent(loss, p0)
    results["Gradient Descent"]    = p_gd
    errors["Gradient Descent"]     = mae(p_gd)
    mse_errors["Gradient Descent"] = mse(p_gd)
    println("Gradient Descent   MAE=$(round(errors["Gradient Descent"],digits=7))")
catch e
    println("Gradient Descent failed: $e")
end

# ── Print fitted constants ──────────────────────────────────────────────────
println("\n=== Fitted Constants ===")
println("Parameter: [X_amp,  ζ,      ωn (rad/s),  ϕ]")
for (name, p) in results
    println("$name")
    println("  X=$(round(p[1],digits=5))  ζ=$(round(p[2],digits=5))  ωn=$(round(p[3],digits=4)) rad/s
    ↪  fn=$(round(p[3]/(2π),digits=3)) Hz  ϕ=$(round(p[4],digits=4)) rad ωd=$(round(p[3]*√(1-p[2]^2),digits=4)) rad/s")
    println("  MAE = $(round(errors[name], digits=7)) m")
end

# ── Plot 1: Fitted curves vs filtered signal ────────────────────────────────
algo_colors = ["CurveFit (auto)" => :steelblue,
               "Gauss-Newton"    => :orange,
               "Gradient Descent"=> :green]

p1 = plot(t_arr, x_arr,
    label  = "Filtered signal",
    color  = :gray, alpha = 0.35, lw = 1.0,
    xlabel = "Time (s)", ylabel = "Displacement (m)",
    title  = "Free Vibration Curve Fit  x(t)=Xe^{-ζωt}cos(ωdt-ϕ)")

for (name, col) in algo_colors
    haskey(results, name) || continue
    plot!(p1, t_arr, fv_model(results[name], t_arr),
        label = "$name  MAE=$(round(errors[name],digits=5))",
        lw = 2.0, color = col)
end

# ── Plot 2 & 3: MAE and MSE bar charts side-by-side ─────────────────────────
names_sorted = sort(collect(keys(errors)))
mae_vals     = [errors[n] for n in names_sorted]
mse_vals     = [mse_errors[n] for n in names_sorted]
bar_cols     = [get(Dict(algo_colors), n, :gray) for n in names_sorted]

p2 = bar(names_sorted, mae_vals,
    xlabel = "Algorithm", ylabel = "MAE (m)",
    title  = "Mean Absolute Error",
    color  = bar_cols, legend = false,
    xtickfontsize = 9)

p3 = bar(names_sorted, mse_vals,
    xlabel = "Algorithm", ylabel = "MSE (m²)",
    title  = "Mean Squared Error",
    color  = bar_cols, legend = false,
```

```
    xtickfontsize = 9)

p_all = plot(p1, p2, p3, layout = @layout([a; b c]), size = (1000, 900))
end

#  8ce2732c-e3b8-4888-9f25-7046b0782d98
md"""
Therfore, we estimate

``X = 0.18813,\zeta=0.01896,w_n = 19.9328, =0.1525``.

where ``w_n = \sqrt{\frac{k}{m}}``.

the value of m and k cant be determined by value.

"""

#  057655cf-c369-4b36-bdfc-cc77ff1a85f4
begin
        const X_amp = 0.18813
        const zeta0 = 0.01896
        const wn    = 19.9328
        const phi   = 0.1525

        # Initial conditions from the fitted curve at t = 0
        wd0  = sqrt(1.0 - zeta0^2) * wn
        x0   = X_amp * cos(phi)
        xd0  = X_amp * (-zeta0 * wn * cos(phi) + wd0 * sin(phi))

        # ── Analytical free-vibration response for arbitrary ζ ──────────────────
        function free_response(t_vec, zeta, wn, x0, xd0)
            if abs(zeta - 1.0) < 1e-6          # critically damped
                A = x0
                B = xd0 + wn * x0
                return @. (A + B * t_vec) * exp(-wn * t_vec)
            elseif zeta < 1.0                  # underdamped
                wd = sqrt(1.0 - zeta^2) * wn
                A  = x0
                B  = (xd0 + zeta * wn * x0) / wd
                return @. exp(-zeta * wn * t_vec) * (A * cos(wd * t_vec) + B * sin(wd * t_vec))
            else                               # overdamped
                sq = sqrt(zeta^2 - 1.0)
                r1 = (-zeta + sq) * wn
                r2 = (-zeta - sq) * wn
                A  = (xd0 - r2 * x0) / (r1 - r2)
                B  = x0 - A
                return @. A * exp(r1 * t_vec) + B * exp(r2 * t_vec)
            end
        end

        # ── Sweep
        ↪ ──────────────────────────────────────────────────────────────────────────────────────────────────────────────
        zeta_vals = [0.0, 0.1, 0.3, 0.5, 0.7, 0.9, 1.0, 1.5, 2.0]
        t_vec     = range(0.0, 2.0, length = 2000)   # 0-2 s captures decay well

        # Color gradient: blue (undamped) → red (overdamped)
        colors = cgrad(:RdYlBu, length(zeta_vals), rev = true)

        # ── Plot
        ↪ ──────────────────────────────────────────────────────────────────────────────────────────────────────────────
        p = plot(
            xlabel = "Time (s)",
            ylabel = "Displacement (m)",
            title  = "Free Vibration - Effect of Damping Ratio (ωn = $(wn) rad/s)",
            legend = :topright,
            size   = (1000, 600),
            lw     = 1.8
        )

        for (i, zeta) in enumerate(zeta_vals)
            x_ = free_response(collect(t_vec), zeta, wn, x0, xd0)

            region = zeta < 1.0 ? "underdamped" :
                     abs(zeta - 1.0) < 1e-6 ? "critically damped" : "overdamped"

            label_str = "ζ = $(zeta)  ($region)"
            plot!(p, t_vec, x_, label = label_str, color = colors[i], lw = 1.8)
        end

        # Mark the original fitted ζ
        x_orig = free_response(collect(t_vec), zeta0, wn, x0, xd0)
        plot!(p, t_vec, x_orig,
            label     = "ζ = $(zeta0)  (fitted)",
            color     = :black,
            lw        = 2.2,
            linestyle = :dash)

end
```

B.3.2 `week2.jl` — 2주차: 가진 응답 및 질량-강성 Pareto

Rotary 엔진의 접선력을 Fourier 분해하여 날개 1DOF 운동방정식의 외력으로 사용한다. `DifferentialEquations.jl`의 `Tsit5` 적분기로 시간 응답을 구하고, 질량–강성 2D grid에 대한 brute-force Pareto frontier를 `Threads.@threads`로 병렬 계산한다. RPM sweep으로 공진 RPM 대역을 확인한 뒤 주파수 분리 조건을 통해 권장 강성 상한을 도출한다.

week2.jl

```
### A Pluto.jl notebook ###
# v0.20.25

using Markdown
using InteractiveUtils

# This Pluto notebook uses @bind for interactivity. When running this notebook outside of Pluto, the following 'mock version' of @bind gives bound variables a default value (instead of an error).
macro bind(def, element)
    #! format: off
    return quote
        local iv = try Base.loaded_modules[Base.PkgId(Base.UUID("6e696c72-6542-2067-7265-42206c756150"), "AbstractPlutoDingetjes")].Bonds.initial_value catch; b -> missing; end
        local el = $(esc(element))
        global $(esc(def)) = Core.applicable(Base.get, el) ? Base.get(el) : iv(el)
        el
    end
    #! format: on
end

#  aa541eea-403b-4b2c-a199-a89d33e1bccf
using Plots, DifferentialEquations,LaTeXStrings,PlutoUI, FFTW,DSP,ForwardDiff

#  ca4b4f26-8f5d-43b3-b681-b6bc64996ad2
include("./resources/week2/tangential_force.jl");

#  37f7f6d2-3ce2-4cf7-b6fa-218e95ef16f6
include("./resources/week2/fourier_torque.jl");

#  a6bbd2e0-5989-11f1-aeb3-0925625a1384
md"""
# WEEK 2
"""

#  35966b7b-3093-4773-95eb-5b01430b16bb
md"""
the typical single pistol engine is radial engine, as shown below
![radial engine]()

since horizontal mode's natural frequency is high enough and less vibration then vertical direction, we only consider vertical vibration in this case. if the motion of engine is stationary enough, neglecting moment force couple and therefore consider only effect by force. For sake of simplicity, if we take equivalent term and assume the total effect by radial engine is same produced by single cylinder.(at least tangentional force, what we are considering most for now)then we can express total base excitation by engine, as given in *Analysis of torsional vibration in internal combustion engines: modelling and experimental validation)*(DOI: 10.1243/14644193JMBD126)
, we can assume the force is like below.
"""

#  190721f2-a1a3-4f36-8c0d-3c5875275364
@bind rpm NumberField(0:2e5, default=2e3)

#  5c236fe5-2a9e-4432-92d4-281b8a7e9cd2
plot_rpm_fig(rpm)

#  09cc22e0-cd14-48ff-b492-655b559eee4c
r = fourier_decompose(rpm, 24); #return rpm based behavior on upper engine design

#  ebc6bbc7-f1eb-4c08-834f-4b1022d76a61
md"""
Based on the graph above, the horizontal force component can be neglected. The vertical force component is then expressed as a sum of sinusoids via Fourier decomposition (order = 24)."""

#  8fb0efcb-eff8-4614-9bbf-73bdb8c9e815
r.plot


#  a4c7e5fe-bea9-414a-9afe-490e06624578
md"""
note n/2 since 4 stroke engine require 2 revolution to finish full cycle """

#  2dd3b37e-511a-4365-8bdb-b3eb5281930b
md"""
Using the Fourier-decomposed excitation ``Y(t)`` derived above and the equation of motion from Week 1,

``m\ddot{x} + c\dot{x} + kx = \sum_{i=1}^{\infty} H(i\omega)\,e^{i\omega t} = Y(t)``

we assume the wing retains the same modal properties as identified in Week 1: ``\zeta = 0.01896``, ``\omega_n = 19.9328`` rad/s.

**For simplicity, the wing mass is fixed at** ``m = 100`` **kg.**
"""

#  54daf206-657f-4fdb-8a70-48f99fd973da
@bind t_last NumberField(0:0.1:30, default=10)

```

```
#  43e20536-6326-4cac-8fea-5874280d2dbb
begin
        # ——— Parameters ———————————————————————————————————————————————————————————————————————————————————————————————————————————
        wn = 19.3828
        m  = 100
        # mass            [kg]
        c  = 0.01896*(2*m*19.9328)   # damping coeff   [N·s/m]
        k  = wn^2*(m)   # stiffness       [N/m]
        y(t) = 0.5*evaluate_fourier(t,r.omegas,r.coefficients) #[N]
        #! divide by half assume aircraft is fixed wing aircraft.

        # ——— Equation of motion: m*x'' + c*x' + k*x = y(t) ——————————————————————————————————————————————————————————————————————
        # State: u = [x, x'] #state space representation
        function eom!(du, u, p, t)
            m, c, k = p
            du[1] = u[2]
            du[2] = (y(t) - c*u[2] - k*u[1]) / m
        end

        u0    = [0.0, 0]          # initial conditions: x(0)=0, x'(0)=0
        tspan = (0.0,t_last)
        p     = (m, c, k)
        prob = ODEProblem(eom!, u0, tspan, p)
        sol  = solve(prob, Tsit5(), reltol=1e-8, abstol=1e-8)

# ——— Plot ———————————————————————————————————————————————————————————————————————————————————————————————————————————————————
t_vec = range(tspan[1], tspan[2], length=2000)
x_vec = [sol(t)[1] for t in t_vec]
y_vec = [y(t)      for t in t_vec]

plt = plot(t_vec, y_vec, label="y(t) [N]", lw=1.5, ls=:dash, color=:orange,
           xlabel="Time [s]", ylabel="Force [N]", legend=:topright,
           title="Damped Forced Vibration  (m=$(m), c=$(round(c)), k=$(round(k)))")
plt2 = twinx(plt)
plot!(plt2, t_vec, x_vec, label="x(t) [m]", lw=2, color=:steelblue,
      ylabel="Displacement [m]", legend=:topleft)

#savefig(ptot,"week2_total.png")
end

#  6fd24617-17c7-4ecc-b1b7-a851a4b0ba49
begin
        plot(t_vec,y_vec,xlims=(0,1),title=(L"Y(t)"))
        #savefig(p_force,"week2_only_force.png")

end

#  9bada243-2fc2-4712-a5da-03c32c8925db
begin
        plot(t_vec,x_vec,title="displacement")
        #savefig(p_disp,"week2_only_displacement.png")

end

#  157801b1-244f-49f0-aee7-514373a308b1
md"""
## Wing Mass vs. Peak Amplitude - Pareto Frontier
"""

#  dc069521-aba4-4d8b-88da-65c0e0cdc89b
md"""
Assuming the natural frequency is held constant, we investigate how the peak steady-state amplitude varies with wing mass.
"""

#  17a17e4c-7059-4da9-9cf4-996b343c005e
begin
        m_range = collect(100:100:15000)
        m_max = Vector{Float64}(undef, length(m_range))
        Threads.@threads for i in eachindex(m_range)
                m_i = m_range[i]
                prob_new = remake(prob; p=(m_i, 0.01896*2*m_i*wn, m_i*wn^2))
                sol_ = solve(prob_new, Tsit5(), reltol=1e-8, abstol=1e-8)
                x_vec_ = [sol_(t)[1] for t in t_vec]
                m_max[i] = maximum(abs.(x_vec_))
        end
end

#  faa9a60f-42c7-423d-9847-4e5491c34b0d
begin
        # Brute-force 2D grid: vary mass AND stiffness independently (not tied via ωn).
        # For each (m, k) pair simulate peak amplitude, then extract Pareto-optimal front.
        ζ_bf      = 0.01896
        m_bf_grid = collect(100:500:15000)
        k_bf_grid = exp.(range(log(5e2), log(8e6), length=30))
        all_combos = collect(Iterators.product(m_bf_grid, k_bf_grid))

        bf_results = Vector{Tuple{Float64,Float64}}(undef, length(all_combos))
        Threads.@threads for idx in eachindex(all_combos)
                m_i, k_i = all_combos[idx]
                c_i = 2ζ_bf * sqrt(m_i * k_i)
```

```
                sol_i = solve(remake(prob; p=(m_i, c_i, k_i)), Tsit5(), reltol=1e-8, abstol=1e-8)
                x_i = [sol_i(t)[1] for t in t_vec]
                bf_results[idx] = (m_i, maximum(abs.(x_i)))
        end

        # A point (m_i, a_i) is Pareto-optimal if no other point (m_j, a_j) has
        # m_j ≤ m_i AND a_j ≤ a_i with at least one strict inequality.
        bf_pareto = filter(bf_results) do (m_i, a_i)
                !any(bf_results) do (m_j, a_j)
                        m_j ≤ m_i && a_j ≤ a_i && (m_j < m_i || a_j < a_i)
                end
        end
        sort!(bf_pareto, by = first)
        bf_m = first.(bf_pareto)
        bf_a = last.(bf_pareto);
end

#  e8a2b4c1-3f5d-4e6b-8a9c-0d1e2f3a4b5c
begin
        # ── Constraints (engineering inputs - set by user) ──────────────────────
        x_limit = 1e-2        # max allowable amplitude [m]  ← adjust to your spec
        m_min   = 100.0       # structural minimum mass  [kg] ← adjust to your spec

        # ── Theoretical 1/m reference calibrated to first data point ────────────
        C_ref = m_max[1] * m_range[1]
        m_ref_line = 10 .^ range(log10(m_range[1]), log10(m_range[end]), length=300)

        plt_pareto = plot(m_range, m_max,
                xlabel="Mass [kg]", ylabel="Max Amplitude [m]",
                xscale=:log10, yscale=:log10,
                title="Wing Mass vs Peak Amplitude - Pareto Frontier",
                label="Simulated frontier", lw=2, color=:steelblue,
                marker=:circle, markersize=2, legend=:topright)

        # 1/m reference
        plot!(plt_pareto, m_ref_line, C_ref ./ m_ref_line,
                ls=:dash, color=:gray, lw=1.5, label="1/m reference (slope = 1)")

        # amplitude constraint (horizontal)
        hline!(plt_pareto, [x_limit],
                ls=:dot, color=:red, lw=2, label="Amplitude limit $(x_limit) m")

        # mass constraint (vertical)
        vline!(plt_pareto, [m_min],
                ls=:dot, color=:forestgreen, lw=2, label="Min. mass $(m_min) kg")

        # baseline design point
        scatter!(plt_pareto, [m_range[1]], [m_max[1]],
                markersize=8, color=:orange, markershape=:star5, label="Baseline (m=$(m_range[1]) kg)")

        # Brute-force Pareto frontier: non-dominated designs from 2D (mass × stiffness) grid
        plot!(plt_pareto, bf_m, bf_a,
                lw=2, color=:crimson, ls=:dashdot,
                marker=:diamond, markersize=4, label="Brute-force Pareto frontier")

        plt_pareto
end

#  2be28692-a16d-4556-a39e-d4400207fde9
md"""
This indicate we need to adjust stiffness to get better system
"""

#  fa3cdf7c-9af5-4251-b4a3-bf581ad8d09c
md"""
## Recommended Stiffness k for Flight-Envelope Safety - Engineering Justification
"""


#  a1b2c3d4-e5f6-4a7b-8c9d-0e1f2a3b4c5d
md"""
### What is the Flight Envelope?

In this context, the **flight envelope** refers to the **RPM range** over which the engine operates during normal flight.

Fundamental excitation frequency of a 4-stroke radial engine: ``\omega_{exc} = \frac{\text{RPM} \cdot \pi}{60}`` [rad/s]

If the wing natural frequency ``\omega_n = \sqrt{k/m}`` coincides with this excitation frequency, **resonance** occurs and the
↪  vibration amplitude grows rapidly.

**Design principle - Frequency Separation (Soft Design):** Choose ``k`` such that ``\omega_n < \eta \cdot \omega_{exc,min}`` holds
↪  across the entire flight envelope. (``\eta = 0.8``, 20% safety margin)
"""

#  b2c3d4e5-f6a7-4b8c-9d0e-1f2a3b4c5d6e
begin
        rpm_flight  = collect(600:50:2700)
        x_peak_rpm  = Vector{Float64}(undef, length(rpm_flight))

        for i in eachindex(rpm_flight)
                r_i    = fourier_decompose(rpm_flight[i], 24)
                oms_i  = copy(r_i.omegas)
```

```
                coef_i = copy(r_i.coefficients)
                yi = let oms = oms_i, coef = coef_i
                        t -> 0.5 * evaluate_fourier(t, oms, coef)
                end
                prob_i = ODEProblem(
                        (du, u, p, t) -> begin
                                m_, c_, k_ = p
                                du[1] = u[2]
                                du[2] = (yi(t) - c_*u[2] - k_*u[1]) / m_
                        end,
                        [0.0, 0.0], (0.0, 20.0), (m, c, k)
                )
                sol_i  = solve(prob_i, Tsit5(), reltol=1e-6, abstol=1e-6)
                t_ss   = range(15.0, 20.0, length=500)
                x_peak_rpm[i] = maximum(abs.([sol_i(t)[1] for t in t_ss]))
        end
end

#  c3d4e5f6-a7b8-4c9d-0e1f-2a3b4c5d6e7f
begin
        x_safe_env = 5e-2   # allowable peak amplitude [m] - adjust to your spec

        plt_env = plot(rpm_flight, x_peak_rpm,
                xlabel="Engine RPM", ylabel="Steady-state Peak Amplitude [m]",
                title="Amplitude Response across Flight Envelope  (k = $(round(Int, k)) N/m)",
                label="Peak displacement |x(t)|", lw=2, color=:steelblue,
                marker=:circle, markersize=3, legend=:topright)

        hline!(plt_env, [x_safe_env],
                ls=:dash, color=:red, lw=2,
                label="Allowable amplitude limit  $(x_safe_env) m")

        resonant_rpm_1st = wn / (π/60)
        vline!(plt_env, [resonant_rpm_1st],
                ls=:dot, color=:orange, lw=2,
                label="1st resonance RPM ≈ $(round(Int, resonant_rpm_1st))")

        plt_env
        savefig(plt_env,"week2_rpm_relative")
end

#  d4e5f6a7-b8c9-4d0e-1f2a-3b4c5d6e7f8a
begin
        rpm_min_fe   = 600.0
        rpm_max_fe   = 2700.0
        η_sep        = 0.8

        ω_exc_min_fe = rpm_min_fe * π / 60
        ω_exc_max_fe = rpm_max_fe * π / 60
        wn_limit_fe  = η_sep * ω_exc_min_fe
        k_rec        = m * wn_limit_fe^2

        (;
                ω_exc_min_rad_s  = round(ω_exc_min_fe, digits=2),
                ω_exc_max_rad_s  = round(ω_exc_max_fe, digits=2),
                wn_limit_rad_s   = round(wn_limit_fe,  digits=2),
                k_recommended    = round(Int, k_rec),
                k_current        = round(Int, k),
                wn_current_rad_s = round(wn, digits=4),
                is_safe          = k <= k_rec
        );
end

#  abfacf7b-e534-43d4-90ef-d008228a9e44
md"""
### Engineering Justification

**1. Flight Envelope Definition**
Operating RPM range for a small piston-engine aircraft: 600 - 2700 RPM
→ Fundamental excitation frequency range: $(round(600*π/60, digits=1)) - $(round(2700*π/60, digits=1)) rad/s


**2. Resonance Avoidance Condition (Frequency Separation)**
``\omega_n < 0.8 \times \omega_{exc,min} = 0.8 \times`` $(round(ω_exc_min_fe, digits=1)) ``\approx`` $(round(wn_limit_fe, digits=1)) rad/s
→ Recommended stiffness upper bound: ``k \leq m \cdot \omega_{n,lim}^2`` = $(round(Int, k_rec)) N/m


**3. Current Design Verification**
$\omega_n$ = $(round(wn, digits=2))rad/s $(k <= k_rec ? "<" : ">") $(round(wn_limit_fe, digits=1))rad/s &&
k = $(round(Int, k)) N/m $(k <= k_rec ? "<" : ">") $(round(Int, k_rec)) N/m

→ $(k <= k_rec ? "** Safe: no resonance occurs across the entire flight envelope**" : "** Unsafe: resonance possible - stiffness must be reduced**")

**4. Conclusion**
Keeping wing stiffness at k ≤ ($(round(Int, k_rec))N/m)

ensures no resonance occurs anywhere in the flight envelope (600 - 2700 RPM). The current design k $\approx$ $(round(Int, k))  N/m satisfies this criterion.


```

```
"""
```

B.3.3 `tangential_force.jl` — 크랭크핀 접선력 모델

단일 실린더 내연기관의 크랭크핀 접선력을 계산하는 라이브러리. Mendes et al.(2008) 논문의 식 (8)–(13)을 구현하며, 가스 하중(`F_tg`)과 왕복 관성력(`F_ta`)의 합으로 결과 접선력 F_t를 산출한다. `sweep_cycle()`은 1회 4행정 사이클(0–720도)에 걸쳐 모든 성분을 벡터로 반환하고, `plot_rpm_fig()`는 임의 RPM에서의 접선력 그래프를 생성해 `week2.jl`의 인터랙티브 슬라이더에 연결된다.

week2/tangential_force.jl

```julia
# ------------------------------------------------------------------------
# tangential_force.jl
#
# Crankpin tangential force and excitation moment for a single cylinder of
# an internal-combustion engine, as a function of crank angle alpha and
# angular velocity omega.
#
# Reference:
#   Mendes A.S., Meirelles P.S., Zampieri D.E. (2008),
#   "Analysis of torsional vibration in internal combustion engines:
#    modelling and experimental validation",
#   Proc. IMechE Vol. 222 Part K (J. Multi-body Dynamics), pp. 155-178.
#
# Implements equations (8)-(13). Target reproduction: Fig. 14
# (crankpin tangential forces at 2000 and 2550 r/min).
#
# Pressure model: parametric Gaussian approximation of Figs. 18 & 19.
# The paper uses MEASURED p(alpha) curves; this is a stand-in only and
# must be replaced with measured data for quantitative agreement.
# ------------------------------------------------------------------------

using Plots

# === Engine geometry & oscillating mass (paper p.156) ======================
const D_PISTON = 0.105        # piston diameter dp [m]
const STROKE   = 0.137        # piston stroke s    [m]
const R_CRANK  = STROKE / 2   # crank radius r     [m]    (s = 2r)
const L_ROD    = 0.207        # con-rod length L   [m]
const M_OSC    = 2.521        # oscillating mass ma (piston + mab) [kg]
const LAMBDA   = R_CRANK / L_ROD          # lambda = r/L  ~ 0.331

rpm_to_omega(n_e_rpm) = 2pi * n_e_rpm / 60.0       # [rad/s]

# === Kinematics ============================================================
"Connecting-rod inclination beta from sin(beta) = lambda * sin(alpha)."
beta_angle(alpha) = asin(LAMBDA * sin(alpha))

"Geometric factor sin(alpha+beta)/cos(beta), shared by eqs (9) and (11)."
function geo_factor(alpha)
    b = beta_angle(alpha)
    return sin(alpha + b) / cos(b)
end

# === Cylinder pressure (parametric Gaussian approximation) =================
# Peak combustion pressure read off Fig. 19 of the paper [Pa]
function p_peak_fig19(n_e_rpm)
    rpm = [1000.0, 1400.0, 1600.0, 1800.0, 2000.0, 2200.0, 2400.0, 2550.0]
    pp  = [14.2,   14.5,   14.5,   14.8,   15.0,   15.0,   13.0,    6.0] .* 1e6
    n = clamp(n_e_rpm, rpm[1], rpm[end])
    i = something(findlast(<=(n), rpm), 1)
    i = min(i, length(rpm) - 1)
    t = (n - rpm[i]) / (rpm[i+1] - rpm[i])
    return pp[i] + t * (pp[i+1] - pp[i])
end

"""
    cylinder_pressure(alpha_rad, n_e_rpm; p_atm, alpha_peak_deg, sigma_deg) -> Pa

Approximate cylinder pressure for one 4-stroke cycle (0..720 deg).
Centre of combustion ~12 deg ATDC (compression TDC = 360 deg by convention
used in Fig. 18). Replace by measured p(alpha) for accurate analyses.
"""
function cylinder_pressure(alpha_rad, n_e_rpm;
                           p_atm=1.0e5,
                           alpha_peak_deg=372.0,
                           sigma_deg=25.0)
    alpha_deg = mod(rad2deg(alpha_rad), 720.0)
    p_peak = p_peak_fig19(n_e_rpm)
    return p_atm + (p_peak - p_atm) *
           exp(-0.5 * ((alpha_deg - alpha_peak_deg) / sigma_deg)^2)
end

# === Forces (eqs 8 - 12) ===================================================
"Gas load on piston, eq. (8): F_g = pi * dp^2 / 4 * p."
gas_load(p) = pi * D_PISTON^2 / 4 * p

"Tangential gas load, eq. (9): F_tg = F_g * sin(alpha+beta)/cos(beta)."
```

```
tangential_gas_load(alpha, p) = gas_load(p) * geo_factor(alpha)

"""
Oscillating inertial force, eq. (10):
F_ia = ma r w^2 ( cos a + lambda cos 2a - (lambda^3/4) cos 4a
                                        + (9 lambda^5 / 128) cos 6a )
"""
function oscillating_inertia(alpha, omega)
    return M_OSC * R_CRANK * omega^2 * (
        cos(alpha)
        + LAMBDA       * cos(2alpha)
        - (LAMBDA^3 / 4)           * cos(4alpha)
        + (9 * LAMBDA^5 / 128)     * cos(6alpha)
    )
end

"Tangential inertial force, eq. (11): F_ta = F_ia * sin(alpha+beta)/cos(beta)."
tangential_inertia(alpha, omega) =
    oscillating_inertia(alpha, omega) * geo_factor(alpha)

"""
    tangential_force(alpha, omega, p) -> (F_tg, F_ta, F_t)

Crankpin tangential forces at crank angle `alpha` [rad] and angular speed
`omega` [rad/s], with cylinder pressure `p` [Pa] at this angle.
Returns gas, inertia and resultant components (eq. 12).
"""
function tangential_force(alpha, omega, p)
    F_tg = tangential_gas_load(alpha, p)
    F_ta = tangential_inertia(alpha, omega)
    return F_tg, F_ta, F_tg + F_ta
end

"Excitation moment for one crank throw, eq. (13): M_t = F_t * r."
moment_from_Ft(F_t) = F_t * R_CRANK

# === Sweep over one 4-stroke cycle ============================================
"""
    sweep_cycle(n_e_rpm; alpha_deg=0:1:720, p_func=cylinder_pressure)

Compute the three tangential force components and the excitation moment
over one 4-stroke cycle at engine speed `n_e_rpm`.
Returns a NamedTuple with vectors `alpha_deg`, `F_tg`, `F_ta`, `F_t`, `M_t`.
"""
function sweep_cycle(n_e_rpm; alpha_deg=0:1:720, p_func=cylinder_pressure)
    omega = rpm_to_omega(n_e_rpm)
    a_rad = deg2rad.(collect(alpha_deg))
    F_tg = similar(a_rad); F_ta = similar(a_rad); F_t = similar(a_rad)
    for i in eachindex(a_rad)
        p_i = p_func(a_rad[i], n_e_rpm)
        F_tg[i], F_ta[i], F_t[i] = tangential_force(a_rad[i], omega, p_i)
    end
    # World-frame decomposition of the tangential force vector.
    # alpha from TDC (cylinder axis), rotation increases alpha.
    # crankpin position    : (r sin a, r cos a)
    # tangent unit vector  : (cos a, -sin a)   (direction of rotation)
    F_t_x =  F_t .* cos.(a_rad)        # horizontal component (sideways)
    F_t_y = -F_t .* sin.(a_rad)        # vertical component   (along cylinder axis, + up)
    return (alpha_deg = collect(alpha_deg),
            F_tg = F_tg, F_ta = F_ta, F_t = F_t,
            F_t_x = F_t_x, F_t_y = F_t_y,
            M_t  = moment_from_Ft.(F_t))
end

# === Plot reproducing Fig. 14 =================================================
function plot_fig14(; savepath::Union{Nothing,String}=nothing)
    r1 = sweep_cycle(2000)
    r2 = sweep_cycle(2550)

    common = (xlabel="Crankshaft angle [deg]", ylabel="[N]",
              xticks=0:60:720, legend=:topleft, lw=1.5,
              framestyle=:box)

    p1 = plot(r1.alpha_deg, r1.F_tg, label="Tangential gas load",
              color=:gray, title="Tangential Forces at 2000 rpm"; common...)
    plot!(p1, r1.alpha_deg, r1.F_ta, label="Tangential inertia force",
          color=:black)
    plot!(p1, r1.alpha_deg, r1.F_t, label="Resultant tangential force",
          color=:black, ls=:dash)

    p2 = plot(r2.alpha_deg, r2.F_tg, label="Tangential gas load",
              color=:gray, title="Tangential Forces at 2550 rpm"; common...)
    plot!(p2, r2.alpha_deg, r2.F_ta, label="Tangential inertia force",
          color=:black)
    plot!(p2, r2.alpha_deg, r2.F_t, label="Resultant tangential force",
          color=:black, ls=:dash)

    fig = plot(p1, p2, layout=(2,1), size=(900, 800))
    savepath !== nothing && savefig(fig, savepath)
    return fig
end
# === plot for interactive query of tangential forces at a single speed ========
```

```
function plot_rpm_fig(speed_1; savepath::Union{Nothing,String}=nothing)
    r1 = sweep_cycle(speed_1)

    common = (xlabel="Crankshaft angle [deg]", ylabel="[N]",
              xticks=0:60:720, legend=:topleft, lw=1.5,
              framestyle=:box)

    p1 = plot(r1.alpha_deg, r1.F_tg, label="Tangential gas load",
              color=:gray, title="Tangential Forces at 2000 rpm"; common...)
    plot!(p1, r1.alpha_deg, r1.F_ta, label="Tangential inertia force",
          color=:black)
    plot!(p1, r1.alpha_deg, r1.F_t, label="Resultant tangential force",
          color=:black, ls=:dash)

    fig = plot(p1, size=(900, 800))
    #savepath !== nothing && savefig(fig, savepath)
    return fig
end

# Run with:
#   julia> include("tangential_force.jl")
#   julia> plot_fig14(savepath="imgs/tangential_force_fig14.png")
#
# Single-point query:
#   julia> tangential_force(deg2rad(380.0), rpm_to_omega(2000),
#                           cylinder_pressure(deg2rad(380.0), 2000))
#=If my sign convention for F_t,y (positive up) is the opposite of what you wanted,
    just flip the sign in line 132 of tangential_force.jl (F_t_y = -F_t .* sin.(a_rad) + F_t .* sin.(a_rad)).=#
```

B.3.4 `fourier_torque.jl` — 접선력의 Fourier 분해

`tangential_force.jl`에서 얻은 F_t 파형을 Fourier 급수로 분해하는 라이브러리. `fourier_decompose()`가 DFT로 복소 계수 H_n과 각주파수 ω_n을 추출하고, `evaluate_fourier()`로 임의의 시간 t에서 재건 파형을 계산한다. 내부 `_stft_spectrogram()` 함수는 3주기 분을 타일링하여 STFT 스펙트로그램을 생성, 시간-주파수 에너지 분포를 시각화한다.

week2/fourier_torque.jl

```
# ------------------------------------------------------------------------------
# fourier_torque.jl
#
# Fourier-series decomposition of the resultant crankpin tangential force
# F_t (and equivalently the per-cylinder excitation torque M_t = F_t * r)
# obtained from `sweep_cycle` in `tangential_force.jl`.
#
# Signal period:  one full 4-stroke cycle = 720 deg crank angle = 2 engine
#                 revolutions, so the fundamental Fourier frequency is
#                 omega_fund = omega_engine / 2.
# This is the same convention as eq. (17) of Mendes et al. (2008):
#
#     M_t(t) = A0/2 + sum_{n=1..N} [ C_n exp(+i n omega_fund t)
#                                  + conj(C_n) exp(-i n omega_fund t) ]
#
# Two-sided complex form (returned by this module):
#
#     F_t(t) = sum_{n = -N..N}  H_n * exp( i * omega_n * t )
#
# with omega_n = n * omega_fund  and  H_{-n} = conj(H_n).
# ------------------------------------------------------------------------------

#include("tangential_force.jl")

using FFTW
using Plots

# --- internal: STFT spectrogram of a periodic signal ------------------------
# Tiles the one-cycle signal `n_cycles` times so the sliding window has room.
# Returns (magnitude, crank_angle_deg_axis, engine_order_axis).
function _stft_spectrogram(sig, n_per_cycle;
                           n_cycles::Integer = 3,
                           win_frac::Real    = 0.5,
                           hop_frac::Real    = 0.025)
    tiled   = repeat(sig, n_cycles)
    N       = length(tiled)
    win_len = max(8, round(Int, n_per_cycle * win_frac))
    hop     = max(1, round(Int, n_per_cycle * hop_frac))
    w       = @. 0.5 * (1 - cos(2pi * (0:win_len-1) / max(win_len - 1, 1)))
    n_win   = (N - win_len) ÷ hop + 1
    nbins   = win_len ÷ 2 + 1
    spec    = zeros(nbins, n_win)
    for k in 0:n_win-1
        s = k * hop + 1
        seg = view(tiled, s:s+win_len-1) .* w
        spec[:, k+1] = abs.(fft(seg))[1:nbins]
    end
    crank_deg = ((0:n_win-1) .* hop .+ (win_len - 1)/2) .* (720.0 / n_per_cycle)
    orders    = (0:win_len÷2) .* (n_per_cycle / (2 * win_len))   # engine orders
    return spec, crank_deg, orders
end
```

```
"""
    fourier_decompose(rpm, order; signal=:F_t_y, n_samples=1440,
                      max_order_display=15) -> NamedTuple

Fourier-decompose one of the per-cycle force/moment traces from
`sweep_cycle(rpm)`.  Valid values for `signal`:
  :F_t_y  -- vertical component of the tangential force (default)
  :F_t_x  -- horizontal component of the tangential force
  :F_t    -- scalar tangential force (magnitude along tangent)
  :M_t    -- excitation moment (= F_t * r)

The cycle is sampled at `n_samples` uniform crank-angle points over
0..720 deg (excluding the duplicate endpoint), the DFT is computed, and
the first `order` harmonics (plus the DC term) are kept.

The returned `plot` field is a 3-panel figure:
  1. Top    -- original signal vs Fourier reconstruction (time domain).
  2. Middle -- one-sided amplitude spectrum |H_n| vs engine order.
  3. Bottom -- STFT spectrogram in dB (signal tiled 3 cycles).

Returned fields
  omegas        :: Vector{Float64}     two-sided frequencies omega_n  [rad/s]
                                       length = 2*order + 1, ordered -N..N
  coefficients  :: Vector{ComplexF64}  Fourier coefficients H_n matching omegas
  omega_fund    :: Float64             fundamental = omega_engine / 2  [rad/s]
  alpha_deg     :: Vector{Float64}     sample crank angles               [deg]
  signal        :: Vector{Float64}     original sampled signal           [N or N*m]
  reconstructed :: Vector{Float64}     Fourier reconstruction at same samples
  spectrogram   :: NamedTuple          (magnitude, crank_deg, orders) from STFT
  plot          :: Plots.Plot          3-panel composite figure

Reconstruction at arbitrary time `t [s]`:
  f_t = real(sum(coefficients .* exp.(1im .* omegas .* t)))
or use the helper `evaluate_fourier(t, omegas, coefficients)`.
"""
function fourier_decompose(rpm::Real, order::Integer;
                           signal::Symbol  = :F_t_y,
                           n_samples::Integer = 1440,
                           max_order_display::Real = 15.0)
    # --- sample one full cycle ------------------------------------------
    alpha_deg = collect(range(0.0, 720.0, length = n_samples + 1))[1:end-1]
    res = sweep_cycle(rpm; alpha_deg = alpha_deg)
    sig = signal === :M_t   ? res.M_t   :
          signal === :F_t_x ? res.F_t_x :
          signal === :F_t_y ? res.F_t_y :
          signal === :F_t   ? res.F_t   :
          error("fourier_decompose: unknown signal $(signal); use :F_t_y, :F_t_x, :F_t, or :M_t")
    N = length(sig)

    omega_engine = rpm_to_omega(rpm)          # [rad/s]
    omega_fund   = omega_engine / 2           # 720 deg period -> half engine speed

    # --- DFT --> complex Fourier coefficients ---------------------------
    Y = fft(sig)
    c = Y ./ N                                # c_n,  n = 0,1,...,N-1
                                              # c_{N-n} = conj(c_n)  for real sig

    nmax = min(order, N ÷ 2)
    c_pos = c[1:nmax+1]                       # n = 0, 1, ..., nmax

    # two-sided arrays, ordered -nmax .. nmax
    omegas = collect(-nmax:nmax) .* omega_fund
    coeffs = ComplexF64[ n >= 0 ? c_pos[n + 1] : conj(c_pos[-n + 1])
                         for n in -nmax:nmax ]

    # --- reconstruct on the same crank-angle grid -----------------------
    t = deg2rad.(alpha_deg) ./ omega_engine   # time [s] at each sample
    recon = [ real(sum(coeffs .* exp.(1im .* omegas .* tt))) for tt in t ]

    # --- STFT spectrogram (tile 3 cycles) -------------------------------
    spec, sg_crank_deg, sg_orders = _stft_spectrogram(sig, N; n_cycles = 3)
    spec_db = 20 .* log10.(spec ./ maximum(spec) .+ 1e-6)
    keep = findall(sg_orders .<= max_order_display)

    # --- plot panels ----------------------------------------------------
    ylab = signal === :M_t   ? "Excitation moment M_t [N m]" :
           signal === :F_t_x ? "F_t horizontal component  F_t,x [N]" :
           signal === :F_t_y ? "F_t vertical component  F_t,y [N]" :
                               "Resultant tangential force F_t [N]"

    lmarg = 12Plots.mm

    t_ms = t .* 1e3                                  # time axis in milliseconds
    ms_per_deg = 60_000.0 / (rpm * 360)              # 1° crank in ms

    fmt(x) = isinteger(x) ? string(Int(x)) :
                            string(round(x, digits = 1))

    # tick positions chosen on the crank-angle grid (multiples of 60°) and
    # labelled with both ms and deg so the user reads the same tick two ways.
    tick_deg_time  = 0:60:720
```

```
    tick_ms_time   = collect(tick_deg_time) .* ms_per_deg
    tick_labels_t  = ["$(fmt(ms)) ms\n$(d)°"
                      for (ms, d) in zip(tick_ms_time, tick_deg_time)]

    p_time = plot(t_ms, sig,
                  label = "Original", color = :black, lw = 1.5,
                  xlabel = "Time [ms]  /  Crank angle [deg]", ylabel = ylab,
                  xlims  = (0, t_ms[end]),
                  xticks = (tick_ms_time, tick_labels_t),
                  framestyle = :box,
                  left_margin = lmarg,
                  bottom_margin = 4Plots.mm,
                  title  = "Fourier reconstruction @ $(rpm) rpm,  order = $(nmax)")
    plot!(p_time, t_ms, recon,
          label = "Fourier (n = 0..$(nmax))",
          color = :red, lw = 1.5, ls = :dash)

    # one-sided amplitude spectrum (engine orders 0, 0.5, 1, ...)
    orders_x = collect(0:nmax) ./ 2
    mags = Float64[ n == 0 ? abs(c_pos[1]) : 2 * abs(c_pos[n + 1])
                    for n in 0:nmax ]
    p_spec = bar(orders_x, mags,
                 label = "|H_n|",
                 xlabel = "Engine order  (n / 2)", ylabel = "Amplitude",
                 color = :steelblue, lw = 0, bar_width = 0.35,
                 xticks = 0:1:max(1, ceil(orders_x[end])),
                 framestyle = :box,
                 left_margin = lmarg,
                 title = "One-sided amplitude spectrum")

    # dual tick labels for the spectrogram (x is crank angle, also show ms)
    sg_lo = first(sg_crank_deg);  sg_hi = last(sg_crank_deg)
    tick_deg_sg = filter(d -> sg_lo <= d <= sg_hi, 0:360:2880)
    tick_labels_sg = ["$(Int(d))°\n$(fmt(d * ms_per_deg)) ms" for d in tick_deg_sg]

    p_sg = heatmap(sg_crank_deg, sg_orders[keep], spec_db[keep, :],
                   xlabel = "Crank angle [deg]  /  Time [ms]   (3 cycles tiled)",
                   ylabel = "Engine order",
                   color  = :viridis, clim = (-60.0, 0.0),
                   xticks = (tick_deg_sg, tick_labels_sg),
                   colorbar_title = "dB (rel. max)",
                   framestyle = :box,
                   left_margin = lmarg,
                   bottom_margin = 4Plots.mm,
                   title = "STFT spectrogram")

    plt = plot(p_time, p_spec, p_sg, layout = (3, 1), size = (960, 1100))

    return (omegas        = omegas,
            coefficients  = coeffs,
            omega_fund    = omega_fund,
            alpha_deg     = alpha_deg,
            signal        = sig,
            reconstructed = recon,
            spectrogram   = (magnitude  = spec,
                             crank_deg  = sg_crank_deg,
                             orders     = sg_orders),
            plot          = plt)
end

"""
    evaluate_fourier(t, omegas, coefficients) -> Float64

Evaluate f(t) = sum_n H_n * exp(i * omega_n * t) at a single time `t [s]`,
returning the real part (the imaginary part is numerical noise for the
Hermitian-symmetric two-sided arrays returned by `fourier_decompose`).
"""
evaluate_fourier(t, omegas, coefficients) =
    real(sum(coefficients .* exp.(1im .* omegas .* t)))

"""
    spectrum(result) -> (orders, magnitudes)

Convenience: convert a `fourier_decompose` result into a single-sided
amplitude spectrum (engine orders vs |2 H_n| for n >= 1, |H_0| for n = 0).
`orders[n+1] = n / 2` because omega_fund = omega_engine / 2.
"""
function spectrum(result)
    nmax = (length(result.omegas) - 1) ÷ 2
    orders = collect(0:nmax) ./ 2                       # engine orders
    mags   = Float64[ n == 0 ? abs(result.coefficients[nmax + 1]) :
                               2 * abs(result.coefficients[nmax + 1 + n])
                      for n in 0:nmax ]
    return orders, mags
end

# --- example ----------------------------------------------------------------
# include("fourier_torque.jl")
# r = fourier_decompose(2000, 24)                  # 24 harmonics, like the paper
# display(r.plot)
# savefig(r.plot, "imgs/fourier_torque_2000rpm.png")
# evaluate_fourier(0.005, r.omegas, r.coefficients)   # F_t at t = 5 ms
```

```
# orders, mags = spectrum(r)
#fourier_decompose(2000, 24)                # vertical component (default)
#fourier_decompose(2000, 24; signal=:F_t_x)  # horizontal component
#fourier_decompose(2000, 24; signal=:F_t)    # scalar tangential
#fourier_decompose(2000, 24; signal=:M_t)    # excitation moment
```

B.4 3주차: 날개의 로우패스 필터 특성과 공명현상 (MATLAB)

2DOF 날개 모델의 주파수 응답을 분석한다. Bode 선도, 임펄스 응답, Schroeder 위상 멀티사인을 이용하여 날개가 저주파 대역에서 큰 이득을 가지는 로우패스 필터 특성을 확인하고 공진 대역폭을 정량적으로 파악한다.

B.4.1 bode_response.m

wing_2dof.mat의 상태공간 모델로부터 주파수 전달함수를 구성하고, 0.1 Hz – 50 Hz 로그 스윕에서 플런지 h와 피치 θ 각각의 Bode 선도를 출력한다.

```
% bode_response.m  --  Bode plot: plunge h and pitch theta vs gust w_g

here = fileparts(mfilename('fullpath'));
load(fullfile(here,'wing_2dof.mat'), 'A','B','C_out','D_out','fn_struct');

sys = ss(A, B, C_out, D_out);
sys.OutputName = {'h','theta'};
sys.InputName  = {'w_g'};

% frequency range spanning both structural modes
f  = logspace(-1, 1.7, 600);          % 0.1 - 50 Hz
om = 2*pi*f;

[mag, phs] = bode(sys, om);
mag_h  = squeeze(mag(1,1,:));   phs_h  = squeeze(phs(1,1,:));
mag_th = squeeze(mag(2,1,:));   phs_th = squeeze(phs(2,1,:));

fh = fn_struct(1);  ft = fn_struct(2);

col_h  = [0.85 0.33 0.1];
col_th = [0.2  0.4  0.8];

figure('Name','Bode Response','Position',[100 100 760 540]);

subplot(2,1,1)
semilogy(f, mag_h,  'Color',col_h,  'LineWidth',1.8, 'DisplayName','h  (plunge)')
hold on
semilogy(f, mag_th, 'Color',col_th, 'LineWidth',1.8, 'DisplayName','\theta  (pitch)')
xline(fh,'--','Color',[.5 .5 .5],'HandleVisibility','off')
xline(ft,'--','Color',[.5 .5 .5],'HandleVisibility','off')
set(gca,'XScale','log')
ylabel('Magnitude  [m/(m/s)], [rad/(m/s)]')
title('Bode Magnitude');  legend('Location','northwest');  grid on

subplot(2,1,2)
plot(f, phs_h,  'Color',col_h,  'LineWidth',1.8, 'DisplayName','h')
hold on
plot(f, phs_th, 'Color',col_th, 'LineWidth',1.8, 'DisplayName','\theta')
xline(fh,'--','Color',[.5 .5 .5],'HandleVisibility','off')
xline(ft,'--','Color',[.5 .5 .5],'HandleVisibility','off')
set(gca,'XScale','log')
xlabel('Frequency  [Hz]');  ylabel('Phase  [deg]')
title('Bode Phase');  legend('Location','southwest');
grid on;
%exportgraphics(gcf, 'week3_bode_response.png', 'Resolution', 300)
```

Listing 3: bode_response.m — Bode 선도

B.4.2 impulse_response.m

상태공간 시스템에 단위 임펄스 돌풍을 가했을 때 플런지와 피치의 자유 응답을 계산한다. 응답 진폭으로부터 고유진동수와 감쇠비를 정성적으로 확인할 수 있다.

```
% impulse_response.m  --  Impulse response: plunge h and pitch theta

here = fileparts(mfilename('fullpath'));
load(fullfile(here,'wing_2dof.mat'), 'A','B','C_out','D_out','fn_struct');

sys = ss(A, B, C_out, D_out);
sys.OutputName = {'h','theta'};
sys.InputName  = {'w_g'};

% time vector: long enough for transient to decay
t_end = 8;
opt = RespConfig;
opt.Amplitude = 100;
[y, t] = step(sys,t_end,opt);     % y: [Nt x 2 x 1]
```

```
%support all type of inputs: step, impulse, arbitrary time series, etc.

h_imp  = y(:,1) * 1000;           % [mm·s]  (impulse has units output·s)
th_imp = rad2deg(y(:,2));          % [deg·s]

fh = fn_struct(1);  ft = fn_struct(2);

col_h  = [0.85 0.33 0.1];
col_th = [0.2  0.4  0.8];

figure('Name','Impulse Response','Position',[100 100 760 420]);

subplot(2,1,1)
plot(t, h_imp,  'Color',col_h,  'LineWidth',1.5)
yline(0,'Color',[.7 .7 .7])
xlabel('t  [s]');  ylabel('h  [mm\cdots]')
title(sprintf('Plunge  (f_h = %.2f Hz,  f_\\theta = %.2f Hz)', fh, ft))
grid on

subplot(2,1,2)
plot(t, th_imp, 'Color',col_th, 'LineWidth',1.5)
yline(0,'Color',[.7 .7 .7])
xlabel('t  [s]');  ylabel('\theta  [deg\cdots]')
title('Pitch')
grid on
exportgraphics(gcf, 'week3_impulse_response.png', 'Resolution', 300)
```

Listing 4: `impulse_response.m` — 임펄스 응답

B.4.3 `multisine_function_generator.m`

Schroeder 위상 멀티사인 신호를 생성하는 함수. 기본 주파수 f_0, 고조파 수 K, RMS 진폭, 샘플링률, 주기 수를 인자로 받아 `timeseries` 객체로 반환한다.

```
function wg_ts = multisine_function_generator(f0, K, rms_amp, fs, nper)
% MULTISINE_FUNCTION_GENERATOR  Generate a Schroeder-phase multisine as timeseries.
%
%   wg_ts = multisine_function_generator()          -- default params
%   wg_ts = multisine_function_generator(f0,K,rms,fs,nper)
%
%   f0      fundamental frequency [Hz]   default 0.25
%   K       number of harmonics          default 80
%   rms_amp desired RMS amplitude [m/s]  default 1.5
%   fs      sample rate [Hz]             default 400
%   nper    number of periods            default 8

if nargin < 1, f0      = 0.25; end
if nargin < 2, K       = 80;   end
if nargin < 3, rms_amp = 1.5;  end
if nargin < 4, fs      = 400;  end
if nargin < 5, nper    = 8;    end

kk    = 1:K;
Ak    = rms_amp / sqrt(K/2) * ones(1, K);   % flat amplitude, RMS = rms_amp
phi_k = -pi * kk .* (kk-1) / K;             % Schroeder phases (low crest factor)

dt   = 1/fs;
N    = round(fs * (1/f0) * nper);
t    = (0:N-1)' * dt;
wg   = sum(Ak .* sin(2*pi*kk.*t + phi_k), 2);

wg_ts = timeseries(wg, t, 'Name','w_g');
wg_ts.TimeInfo.Units = 's';
end
```

Listing 5: `multisine_function_generator.m` — Schroeder 멀티사인 생성

B.4.4 `multisine_analysis.m`

멀티사인 돌풍 신호에 대한 시간 응답과 Bode 선도를 2×3 서브플롯으로 한 화면에 출력한다. 상단 행은 $w_g(t)$, $h(t)$, $\theta(t)$ 시계열, 하단 행은 $|H_h(f)|$, $|H_\theta(f)|$, 위상을 나타낸다.

```
% multisine_analysis.m  --  multisine time response + bode (2×3 figure)
%
% Row 1: time traces  -- w_g(t)  |  h(t) [mm]  |  θ(t) [deg]
% Row 2: bode         -- |H_h(f)|              |  |H_θ(f)|  |  phase (both)
%! ALL IN ONE: multisine time response + bode (2×3 figure)

here = fileparts(mfilename('fullpath'));
load(fullfile(here,'wing_2dof.mat'), 'A','B','C_out','D_out','fn_struct');

sys = ss(A, B, C_out, D_out);

% ---------- input signal ----------------------------------------
wg_ts = multisine_function_generator();
```

```
t     = wg_ts.Time;
wg    = wg_ts.Data;

% ---------- simulate -- lsim accepts continuous sys directly ------
y_out  = lsim(sys, wg, t);
h_mm   = y_out(:,1) * 1000;
th_deg = rad2deg(y_out(:,2));

fprintf('Plunge RMS = %.3f mm\n',  rms(h_mm));
fprintf('Pitch  RMS = %.4f deg\n', rms(th_deg));

% ---------- bode curves ----------------------------------------
f  = logspace(-1, 1.7, 600);
om = 2*pi*f;
[mag, phs] = bode(sys, om);
mag_h  = squeeze(mag(1,1,:));   phs_h  = squeeze(phs(1,1,:));
mag_th = squeeze(mag(2,1,:));   phs_th = squeeze(phs(2,1,:));

fh = fn_struct(1);  ft = fn_struct(2);

% ---------- zoom to last 2 periods for time plots ---------------
Tper = 1/0.25;
i_z  = t >= t(end) - 2*Tper;
tz   = t(i_z);

col_h  = [0.85 0.33 0.1];
col_th = [0.2  0.4  0.8];
col_in = [0.2  0.4  0.8];

% ---------- 2×3 figure -----------------------------------------
figure('Name','Multisine Analysis','Position',[80 80 1200 620]);

% --- row 1: time ------------------------------------------------
subplot(2,3,1)
plot(tz, wg(i_z), 'Color',col_in, 'LineWidth',0.8)
xlabel('t [s]');  ylabel('w_g [m/s]');  title('INPUT: gust');    grid on

subplot(2,3,2)
plot(tz, h_mm(i_z),   'Color',col_h,  'LineWidth',1.2)
xlabel('t [s]');  ylabel('h [mm]');    title('TIME: plunge');    grid on

subplot(2,3,3)
plot(tz, th_deg(i_z), 'Color',col_th, 'LineWidth',1.2)
xlabel('t [s]');  ylabel('\theta [deg]'); title('TIME: pitch');  grid on

% --- row 2: bode ------------------------------------------------
subplot(2,3,4)
semilogy(f, mag_h, 'Color',col_h, 'LineWidth',1.8)
xline(fh,'--','Color',[.6 .6 .6]);  xline(ft,'--','Color',[.6 .6 .6])
set(gca,'XScale','log')
xlabel('f [Hz]');  ylabel('|H_h| [m/(m/s)]');  title('BODE: plunge mag');  grid on

subplot(2,3,5)
semilogy(f, mag_th, 'Color',col_th, 'LineWidth',1.8)
xline(fh,'--','Color',[.6 .6 .6]);  xline(ft,'--','Color',[.6 .6 .6])
set(gca,'XScale','log')
xlabel('f [Hz]');  ylabel('|H_\theta| [rad/(m/s)]');  title('BODE: pitch mag');  grid on

subplot(2,3,6)
plot(f, phs_h,  'Color',col_h,  'LineWidth',1.8, 'DisplayName','h')
hold on
plot(f, phs_th, 'Color',col_th, 'LineWidth',1.8, 'DisplayName','\theta')
xline(fh,'--','Color',[.6 .6 .6],'HandleVisibility','off')
xline(ft,'--','Color',[.6 .6 .6],'HandleVisibility','off')
set(gca,'XScale','log')
xlabel('f [Hz]');  ylabel('Phase [deg]');  title('BODE: phase');
legend('Location','southwest');  grid on
exportgraphics(gcf, 'week3_multisine_response.png', 'Resolution', 300)
```

Listing 6: `multisine_analysis.m` — 멀티사인 응답 · Bode 통합 분석

B.4.5 `multisine_toolbox_response.m`

MATLAB Control System Toolbox의 `lsim`을 직접 사용하여 멀티사인 돌풍에 대한 응답을 검증한다.

```
% multisine_toolbox_response.m

here = fileparts(mfilename('fullpath'));
load(fullfile(here,'wing_2dof.mat'), 'A','B','C_out','D_out','fn_struct');

sys = ss(A, B, C_out, D_out);

% ---------- multisine design -----------------------------------
f0  = 0.25;  kk = 1:80;  fk = kk * f0;   K = length(kk);
Ak  = 1.5 / sqrt(K/2) * ones(1, K);       % flat, RMS = 1.5 m/s
phi_k = -pi * kk .* (kk-1) / K;           % Schroeder phases

fs = 400;  dt = 1/fs;  Tper = 1/f0;  nper = 8;
N  = round(fs * Tper * nper);
```

```
t  = (0:N-1)' * dt;
wg = sum(Ak .* sin(2*pi*fk.*t + phi_k), 2);

% ---------- simulate with lsim (ZOH) ----------------------------
sysd  = c2d(sys, dt, 'zoh');
y_out = lsim(sysd, wg, t);        % [N x 2]: h, theta

fprintf('Plunge RMS = %.3f mm\n',  rms(y_out(:,1))*1000);
fprintf('Pitch  RMS = %.4f deg\n', rad2deg(rms(y_out(:,2))));

% ---------- bode: continuous curve + values at excited lines -----
fc      = logspace(-1, 1.5, 600);
[Mc, ~] = bode(sys, 2*pi*fc);       % continuous analytical curve
[Mk, ~] = bode(sys, 2*pi*fk);       % at each multisine line

Hh_c = squeeze(Mc(1,1,:));          % plunge channel, continuous
Hh_k = squeeze(Mk(1,1,:))';         % plunge channel, at fk  [1xK]

% ---------- amplitudes directly from bode + design ---------------
Yamp = Hh_k .* Ak * 1000;           % output amplitude [mm] = |H|*|U|

% ---------- natural frequencies ----------------------------------
fh_nat = fn_struct(1);
ft_nat = fn_struct(2);

% ---------- 3-panel figure ---------------------------------------
figure('Name','Multisine FRF (Toolbox)','Position',[100 100 1150 380]);

subplot(1,3,1)
stem(fk, Ak, 'filled', 'MarkerSize', 3, 'Color', [0.2 0.4 0.8])
xlim([0 21]);  xlabel('f [Hz]');  ylabel('|w_g| [m/s]')
title('INPUT: multisine (flat)');  grid on

subplot(1,3,2)
semilogy(fc, Hh_c, 'Color',[0.85 0.33 0.1], 'LineWidth',1.8, ...
         'DisplayName','bode (analytical)')
hold on
scatter(fk, Hh_k, 20, [0.1 0.1 0.5], 'filled', ...
        'DisplayName','bode at f_k')
xline(fh_nat,'--','Color',[.5 .5 .5],'HandleVisibility','off')
xline(ft_nat,'--','Color',[.5 .5 .5],'HandleVisibility','off')
set(gca,'XScale','log');  xlabel('f [Hz]');  ylabel('|H_h(f)|')
title('FILTER: bode magnitude');  legend('Location','southwest');  grid on

subplot(1,3,3)
stem(fk, Yamp, 'filled', 'MarkerSize', 3, 'Color', [0.1 0.1 0.1])
xline(fh_nat,'--','Color',[.5 .5 .5])
xline(ft_nat,'--','Color',[.5 .5 .5])
xlim([0 21]);  xlabel('f [Hz]');  ylabel('|h| [mm]')
title('OUTPUT: response lines');  grid on

out_dir = fullfile(here,'..','imgs');
if ~isfolder(out_dir), mkdir(out_dir); end
saveas(gcf, fullfile(out_dir,'multisine_toolbox_response.png'));
fprintf('Saved.\n');
```

Listing 7: `multisine_toolbox_response.m` — Toolbox 기반 멀티사인 응답

B.5 4주차: 선형 · 비선형 스프링 거동 비교 (MATLAB)

스프링에 3차 비선형 항($k_{nl}x^3$)을 추가한 경우와 선형 모델의 응답 차이를 탐구한다. FAA 이산 돌풍 입력과 멀티사인 입력에 대해 ode45로 수치적분하고, 등가 루트 굽힘 모멘트를 지표로 선형 · 비선형 시스템을 정량 비교한다.

B.5.1 `impulse_function_generator.m`

FAA 14 CFR §25.341(a) 이산 조율 돌풍(1-cosine 파형)을 생성하는 함수. 돌풍 구배 H를 스윕하여 최대 응답을 유발하는 임계 H를 탐색하고 결과를 출력한다.

```
function gust = impulse_function_generator(varargin)
% impulse_function_generator  Certification discrete tuned (1-cosine) gust input.
%   Generates the FAA 14 CFR 25.341(a) / EASA CS-25.341(a) discrete gust as a
%   gust-velocity signal w_g(t) [m/s] for the Week-3 2-DOF wing model, sweeps the
%   gust gradient H to find the critical response, and plots the result.
%   Rationale, formulae, parameters and references: impulse_function_generator.md
%
%   gust = impulse_function_generator('standard','scaled')   % default
%   gust = impulse_function_generator('standard','cs25')     % literal CS-25 range

%   only crit gusts visualized by default, so that the plot is not too busy. For more, run impulse_all.m

cfg = default_config();
for i = 1:2:numel(varargin)
    cfg.(varargin{i}) = varargin{i+1};
end

model         = load_model(cfg.model_file);
```

```
[H, f_gust]   = gradient_sweep(cfg, model);
Uds           = design_gust_velocity(cfg, H, model.U);
[t, wg]       = build_gust_family(H, Uds, model.U, cfg.fs, cfg.settle);
resp          = evaluate_response(model, t, wg);
crit          = find_critical(H, resp);
gust          = pack_output(t, wg, H, Uds, f_gust, resp, crit, cfg);

if cfg.plot
    make_plots(gust, model);
end
end

% =========================================================================

function cfg = default_config()
cfg.standard   = 'scaled';
cfg.model_file = '';
cfg.fs         = 2000;
cfg.nH         = 25;
cfg.settle     = 6;
cfg.Fg         = 1.0;
cfg.Uref       = [];
cfg.band       = [0.5 2.0];
cfg.plot       = true;
end

% -------------------------------------------------------------------------

function model = load_model(model_file)
if isempty(model_file)
    here = fileparts(mfilename('fullpath'));
    cand = {fullfile(here,'wing_2dof.mat')};
    for c = cand
        if isfile(c{1}), model_file = c{1}; break; end
    end
    if isempty(model_file)
        error('impulse_function_generator:model', ...
              'wing_2dof.mat not found next to script or in ../week3.');
    end
end
model = load(model_file, 'A','B','C_out','D_out','U','fn_struct');
end

% -------------------------------------------------------------------------

function [H, f_gust] = gradient_sweep(cfg, model)
switch lower(cfg.standard)
    case 'cs25'
        H = logspace(log10(9.1), log10(107), cfg.nH);
    case 'scaled'
        fn   = model.fn_struct(:)';
        f_lo = cfg.band(1) * min(fn);
        f_hi = cfg.band(2) * max(fn);
        H    = logspace(log10(model.U/(2*f_hi)), log10(model.U/(2*f_lo)), cfg.nH);
    otherwise
        error('impulse_function_generator:standard', ...
              'cfg.standard must be ''scaled'' or ''cs25''.');
end
f_gust = model.U ./ (2*H);
end

% -------------------------------------------------------------------------

function Uds = design_gust_velocity(cfg, H, U)
Uref = cfg.Uref;
if isempty(Uref)
    switch lower(cfg.standard)
        case 'cs25',   Uref = 17.07;
        case 'scaled', Uref = 0.10 * U;
    end
end
Uds = Uref * cfg.Fg .* (H/107).^(1/6);
end

% -------------------------------------------------------------------------

function [t, wg] = build_gust_family(H, Uds, U, fs, settle)
Tg   = 2*H/U;
tmax = max(Tg) + settle;
t    = (0:1/fs:tmax)';
wg   = zeros(numel(t), numel(H));
for j = 1:numel(H)
    in       = t <= Tg(j);
    wg(in,j) = (Uds(j)/2) * (1 - cos(pi*U*t(in)/H(j)));
end
end

% -------------------------------------------------------------------------

function resp = evaluate_response(model, t, wg)
sys          = ss(model.A, model.B, model.C_out, model.D_out);
nH           = size(wg,2);
```

```
resp.h_peak  = zeros(nH,1);
resp.th_peak = zeros(nH,1);
for j = 1:nH
    y               = lsim(sys, wg(:,j), t);
    resp.h_peak(j)  = max(abs(y(:,1))) * 1000;
    resp.th_peak(j) = max(abs(y(:,2))) * 180/pi;
end
end

% ------------------------------------------------------------------------

function crit = find_critical(H, resp)
[crit.h_max,  ih] = max(resp.h_peak);
[crit.th_max, it] = max(resp.th_peak);
crit.H_h   = H(ih);  crit.idx_h  = ih;
crit.H_th  = H(it);  crit.idx_th = it;
end

% ------------------------------------------------------------------------

function gust = pack_output(t, wg, H, Uds, f_gust, resp, crit, cfg)
gust.t      = t;
gust.wg     = wg;
gust.H      = H(:);
gust.Uds    = Uds(:);
gust.f_gust = f_gust(:);
gust.resp   = resp;
gust.crit   = crit;
gust.cfg    = cfg;
end

% ------------------------------------------------------------------------

function make_plots(gust, model)
col_h  = [0.85 0.33 0.10];
col_th = [0.20 0.40 0.80];
sys    = ss(model.A, model.B, model.C_out, model.D_out);

figure('Name','Discrete Tuned Gust (25.341)','Position',[100 100 820 780]);

subplot(3,1,1)
show = unique(round(linspace(1, numel(gust.H), 5)));
hold on
for j = show
    plot(gust.t, gust.wg(:,j), 'LineWidth',1.1, ...
        'DisplayName',sprintf('H = %.2f m', gust.H(j)));
end
plot(gust.t, gust.wg(:,gust.crit.idx_h), 'k', 'LineWidth',2, ...
    'DisplayName','critical (h)');
xlabel('t  [s]');  ylabel('w_g  [m/s]')
title('1-cosine discrete gust family   $U(s) = (U_{ds}/2)\cdot[1-\cos(\pi s/H)]$','Interpreter','latex');
legend('Location','northeast');  grid on;  xlim([0 max(gust.t)])

subplot(3,1,2)
yyaxis left
plot(gust.H, gust.resp.h_peak, '-o', 'Color',col_h, 'MarkerSize',3);  ylabel('peak |h|  [mm]')
yyaxis right
plot(gust.H, gust.resp.th_peak, '-s', 'Color',col_th, 'MarkerSize',3); ylabel('peak |\theta|  [deg]')
xlabel('gust gradient  H  [m]')
title(sprintf('Tuning sweep -- critical  $H_h$ = %.2f m,   $H_{\\theta}$ = %.2f m', ...
    gust.crit.H_h, gust.crit.H_th),'Interpreter','latex');
xline(gust.crit.H_h, '--','Color',col_h, 'HandleVisibility','off');
xline(gust.crit.H_th,'--','Color',col_th,'HandleVisibility','off');
grid on

subplot(3,1,3)
y = lsim(sys, gust.wg(:,gust.crit.idx_h), gust.t);
yyaxis left
plot(gust.t, y(:,1)*1000, 'Color',col_h, 'LineWidth',1.5);  ylabel('h  [mm]')
yyaxis right
plot(gust.t, rad2deg(y(:,2)), 'Color',col_th, 'LineWidth',1.5); ylabel('\theta  [deg]')
xlabel('t  [s]')
title('Response to the critical gust');  grid on
end
```

Listing 8: `impulse_function_generator.m` — FAA 이산 돌풍 생성

B.5.2 `multisine_function_generator.m`

3주차와 동일한 Schroeder 위상 멀티사인 생성기의 4주차 복사본. `nonlinear_response.m`이 멀티사인 입력을 구성할 때 호출한다.

```
function wg_ts = multisine_function_generator(f0, K, rms_amp, fs, nper)
% MULTISINE_FUNCTION_GENERATOR  Generate a Schroeder-phase multisine as timeseries.
%
%   wg_ts = multisine_function_generator()          -- default params
%   wg_ts = multisine_function_generator(f0,K,rms,fs,nper)
%
%   f0      fundamental frequency [Hz]   default 0.25
```

```
%   K       number of harmonics          default 80
%   rms_amp desired RMS amplitude [m/s]  default 1.5
%   fs      sample rate [Hz]             default 400
%   nper    number of periods            default 8

if nargin < 1, f0      = 0.25; end
if nargin < 2, K       = 80;   end
if nargin < 3, rms_amp = 1.5;  end
if nargin < 4, fs      = 400;  end
if nargin < 5, nper    = 8;    end

kk    = 1:K;
Ak    = rms_amp / sqrt(K/2) * ones(1, K);   % flat amplitude, RMS = rms_amp
phi_k = -pi * kk .* (kk-1) / K;             % Schroeder phases (low crest factor)

dt    = 1/fs;
N     = round(fs * (1/f0) * nper);
t     = (0:N-1)' * dt;
wg    = sum(Ak .* sin(2*pi*kk.*t + phi_k), 2);

wg_ts = timeseries(wg, t, 'Name','w_g');
wg_ts.TimeInfo.Units = 's';
end
```

Listing 9: `multisine_function_generator.m` (4주차) — Schroeder 멀티사인 생성

B.5.3 `nonlinear_response.m`

`wing_nl.mat`을 불러와 비선형 스프링 운동방정식을 `ode45`로 수치 적분한다. 입력 인덱스로 임펄스·멀티사인·계단·돌풍 생성기를 선택할 수 있으며, 초기 변위 및 스프링 상수를 파라미터로 지정한다.

```
function out = nonlinear_response(input_idx, h0, theta0, k1)
% nonlinear_response  Time response of the 2-DOF wing with nonlinear springs.
%
%   out = nonlinear_response(input_idx, h0, theta0, k1)
%       input_idx : 1 = impulse | 2 = multisine | 3 = unit step | 4 = gust generator
%       h0        : initial plunge   [m]    (default 0)
%       theta0    : initial pitch    [rad]  (default 0)
%       k1        : spring-1 stiffness [N/m] (default: value in wing_nl.mat)
%
%   Solves   M*q'' + C*q' + Ka*q + g(q) = Fgust*u(t)
%   with     g(q) the per-spring (linear + cubic) structural restoring force,
%            u(t) built from the chosen input by to_forcing (the "input_f_modi").
%
%   Plots a 4-row figure:  input u(t) | h(t) | theta(t) | k1(t) tangent stiffness.
%   When knl1 = knl2 = 0 the restoring is linear and k1(t) is flat at k1.

if nargin < 1, input_idx = 2;  end
if nargin < 2, h0     = 0;     end
if nargin < 3, theta0 = 0;     end
if nargin < 4, k1     = [];    end           % [] -> keep wing_nl.mat value

% ---- fixed run settings ------------------------------------------
t_end = 8;        % simulation length      [s]
fs    = 2000;     % sample rate for u(t)   [Hz]
mag   = 1.0;      % magnitude (step / impulse)
t0    = 0.05;     % onset time             [s]

P = load_plant(k1);                          % matrices + spring params
u = to_forcing(input_idx, t_end, fs, mag, t0); % input_f_modi: u(t) interpolant

% ---- nonlinear equation of motion --------------------------------
g   = @(q) spring_force(q, P);
rhs = @(t,x) [ x(3:4);
               P.M \ ( P.Fgust*u(t) - P.C*x(3:4) - P.Ka*x(1:2) - g(x(1:2)) ) ];

x0   = [h0; theta0; 0; 0];
opts = odeset('RelTol',1e-7, 'AbsTol',1e-9, 'MaxStep',5/fs);
[t, x] = ode45(rhs, [0 t_end], x0, opts);

out = pack(t, x, u, input_idx, P);
plot_response(out);
end

% ========================================================================

function P = load_plant(k1)
here = fileparts(mfilename('fullpath'));
f = fullfile(here, 'wing_nl.mat');
if ~isfile(f)
    error('nonlinear_response:model', 'wing_nl.mat not found -- run build_ss_nl.m first.');
end
S = load(f);
P.M = S.M_mat;  P.C = S.C_mat;  P.Ka = S.Ka;  P.Fgust = S.Fgust;
P.r1 = S.r1;    P.r2 = S.r2;
if isempty(k1), k1 = S.k1; end
P.k1 = k1;      P.k2 = S.k2;
P.knl1 = S.knl1; P.knl2 = S.knl2;
end
```

```
% ------------------------------------------------------------------------

function f = spring_force(q, P)
d1 = q(1) + P.r1*q(2);                          % spring-1 deflection
d2 = q(1) + P.r2*q(2);                          % spring-2 deflection
F1 = P.k1*d1 + P.knl1*d1^3;
F2 = P.k2*d2 + P.knl2*d2^3;
f  = [ F1 + F2;
       P.r1*F1 + P.r2*F2 ];                     % generalised force [h; theta]
end

% ------------------------------------------------------------------------

function u = to_forcing(choice, t_end, fs, mag, t0)
% Convert any input choice into ONE uniform object: u(t) interpolant.
tg = (0:1/fs:t_end)';

switch choice
    case 1                                      % impulse: short pulse ~ Dirac, peak = mag
        w  = 5/fs;
        ug = mag * (tg >= t0 & tg < t0 + w);

    case 2                                      % multisine (Schroeder phases), as Week 3
        f0 = 0.25;  K = 80;  kk = 1:K;
        amp = 1.5/sqrt(K/2);
        phi = -pi*kk.*(kk-1)/K;
        ug  = sum(amp .* sin(2*pi*f0*(tg*kk) + phi), 2);

    case 3                                      % unit step, arbitrary magnitude
        ug = mag * (tg >= t0);

    case 4                                      % certification gust (critical column)
        g   = impulse_function_generator('fs',fs, 'plot',false);
        sel = g.crit.idx_h;
        Fg  = griddedInterpolant(g.t, g.wg(:,sel), 'linear','nearest');
        ug  = Fg(tg);

    otherwise
        error('nonlinear_response:input', 'input_idx must be 1..4.');
end

u = griddedInterpolant(tg, ug, 'linear','nearest');
end

% ------------------------------------------------------------------------

function out = pack(t, x, u, choice, P)
names    = {'impulse','multisine','step','gust-generator'};
out.t     = t;
out.h     = x(:,1);
out.theta = x(:,2);
out.u     = u(t);
out.input = names{choice};

delta1    = out.h + P.r1*out.theta;             % spring-1 deflection history
out.k1eff = P.k1 + 3*P.knl1*delta1.^2;          % instantaneous tangent stiffness
out.P     = P;
end

% ------------------------------------------------------------------------

function plot_response(out)
col_u  = [0.40 0.40 0.40];
col_h  = [0.85 0.33 0.10];
col_th = [0.20 0.40 0.80];
col_k  = [0.30 0.60 0.30];

figure('Name','Nonlinear time response','Position',[100 100 780 860]);

subplot(4,1,1)
plot(out.t, out.u, 'Color',col_u, 'LineWidth',1.4)
ylabel('u(t)  [m/s]');  grid on
title(sprintf('Input: %s', out.input))

subplot(4,1,2)
plot(out.t, out.h*1000, 'Color',col_h, 'LineWidth',1.5)
yline(0,'Color',[.7 .7 .7]);  ylabel('h  [mm]');  grid on

subplot(4,1,3)
plot(out.t, rad2deg(out.theta), 'Color',col_th, 'LineWidth',1.5)
yline(0,'Color',[.7 .7 .7]);  ylabel('\theta  [deg]');  grid on

subplot(4,1,4)
plot(out.t, out.k1eff, 'Color',col_k, 'LineWidth',1.5)
ylabel('k_1(t)  [N/m]');  xlabel('t  [s]');  grid on
title('Spring-1 tangent stiffness  k_1 + 3 k_{nl1}\delta_1^2')
end
```

Listing 10: `nonlinear_response.m` — 비선형 스프링 시간 응답 (ode45)

B.5.4 impulse_all.m

선형(lsim)과 비선형(ode45) 모델에 동일한 임펄스·멀티사인 돌풍을 가하여 플런지·피치 응답을 나란히 비교하는 종합 스크립트.

```
function gust = impulse_function_generator(varargin)
% impulse_function_generator  Certification discrete tuned (1-cosine) gust input.
%   Generates the FAA 14 CFR 25.341(a) / EASA CS-25.341(a) discrete gust as a
%   gust-velocity signal w_g(t) [m/s] for the Week-3 2-DOF wing model, sweeps the
%   gust gradient H to find the critical response, and plots the result.
%   Rationale, formulae, parameters and references: impulse_function_generator.md
%
%   gust = impulse_function_generator('standard','scaled')   % default
%   gust = impulse_function_generator('standard','cs25')     % literal CS-25 range

cfg = default_config();
for i = 1:2:numel(varargin)
    cfg.(varargin{i}) = varargin{i+1};
end

model          = load_model(cfg.model_file);
[H, f_gust]    = gradient_sweep(cfg, model);
Uds            = design_gust_velocity(cfg, H, model.U);
[t, wg]        = build_gust_family(H, Uds, model.U, cfg.fs, cfg.settle);
resp           = evaluate_response(model, t, wg);
crit           = find_critical(H, resp);
gust           = pack_output(t, wg, H, Uds, f_gust, resp, crit, cfg);

if cfg.plot
    make_plots(gust, model);
end
end

% ========================================================================

function cfg = default_config()
cfg.standard   = 'scaled';
cfg.model_file = '';
cfg.fs         = 2000;
cfg.nH         = 25;
cfg.settle     = 6;
cfg.Fg         = 1.0;
cfg.Uref       = [];
cfg.band       = [0.5 2.0];
cfg.plot       = true;
end

% ------------------------------------------------------------------------

function model = load_model(model_file)
if isempty(model_file)
    here = fileparts(mfilename('fullpath'));
    cand = {fullfile(here,'wing_2dof.mat'), ...
            fullfile(here,'..','week3','wing_2dof.mat')};
    for c = cand
        if isfile(c{1}), model_file = c{1}; break; end
    end
    if isempty(model_file)
        error('impulse_function_generator:model', ...
              'wing_2dof.mat not found next to script or in ../week3.');
    end
end
model = load(model_file, 'A','B','C_out','D_out','U','fn_struct');
end

% ------------------------------------------------------------------------

function [H, f_gust] = gradient_sweep(cfg, model)
switch lower(cfg.standard)
    case 'cs25'
        H = logspace(log10(9.1), log10(107), cfg.nH);
    case 'scaled'
        fn   = model.fn_struct(:)';
        f_lo = cfg.band(1) * min(fn);
        f_hi = cfg.band(2) * max(fn);
        H    = logspace(log10(model.U/(2*f_hi)), log10(model.U/(2*f_lo)), cfg.nH);
    otherwise
        error('impulse_function_generator:standard', ...
              'cfg.standard must be ''scaled'' or ''cs25''.');
end
f_gust = model.U ./ (2*H);
end

% ------------------------------------------------------------------------

function Uds = design_gust_velocity(cfg, H, U)
Uref = cfg.Uref;
if isempty(Uref)
    switch lower(cfg.standard)
        case 'cs25',    Uref = 17.07;
        case 'scaled', Uref = 0.10 * U;
```

```
    end
end
Uds = Uref * cfg.Fg .* (H/107).^(1/6);
end

% ------------------------------------------------------------------------

function [t, wg] = build_gust_family(H, Uds, U, fs, settle)
Tg   = 2*H/U;
tmax = max(Tg) + settle;
t    = (0:1/fs:tmax)';
wg   = zeros(numel(t), numel(H));
for j = 1:numel(H)
    in       = t <= Tg(j);
    wg(in,j) = (Uds(j)/2) * (1 - cos(pi*U*t(in)/H(j)));
end
end

% ------------------------------------------------------------------------

function resp = evaluate_response(model, t, wg)
sys          = ss(model.A, model.B, model.C_out, model.D_out);
nH           = size(wg,2);
resp.h_all   = zeros(numel(t), nH);   % plunge histories, all gusts [mm]
resp.th_all  = zeros(numel(t), nH);   % pitch  histories, all gusts [deg]
resp.h_peak  = zeros(nH,1);
resp.th_peak = zeros(nH,1);
for j = 1:nH
    y                = lsim(sys, wg(:,j), t);
    resp.h_all(:,j)  = y(:,1) * 1000;
    resp.th_all(:,j) = y(:,2) * 180/pi;
    resp.h_peak(j)   = max(abs(resp.h_all(:,j)));
    resp.th_peak(j)  = max(abs(resp.th_all(:,j)));
end
end

% ------------------------------------------------------------------------

function crit = find_critical(H, resp)
[crit.h_max,  ih] = max(resp.h_peak);
[crit.th_max, it] = max(resp.th_peak);
crit.H_h   = H(ih);  crit.idx_h  = ih;
crit.H_th  = H(it);  crit.idx_th = it;
end

% ------------------------------------------------------------------------

function gust = pack_output(t, wg, H, Uds, f_gust, resp, crit, cfg)
gust.t      = t;
gust.wg     = wg;
gust.H      = H(:);
gust.Uds    = Uds(:);
gust.f_gust = f_gust(:);
gust.resp   = resp;
gust.crit   = crit;
gust.cfg    = cfg;
end

% ------------------------------------------------------------------------

function make_plots(gust, model) %#ok<INUSD>
nH   = numel(gust.H);
show = unique(round(linspace(1, nH, 5)));   % representative gradients (same set in all panels)
pal  = lines(numel(show));                  % one colour per shown gust, shared across panels

figure('Name','Discrete Tuned Gust (25.341)','Position',[100 100 820 800]);

ax1 = subplot(3,1,1);
draw_family(ax1, gust.t, gust.wg,         show, pal, gust.H, gust.crit.idx_h, ...
    'w_g  [m/s]', '1-cosine discrete gust family   (critical in bold)', true);

ax2 = subplot(3,1,2);
draw_family(ax2, gust.t, gust.resp.h_all,  show, pal, gust.H, gust.crit.idx_h, ...
    'h  [mm]', sprintf('Plunge response   (critical H_h = %.2f m)', gust.crit.H_h), false);

ax3 = subplot(3,1,3);
draw_family(ax3, gust.t, gust.resp.th_all, show, pal, gust.H, gust.crit.idx_th, ...
    '\theta  [deg]', sprintf('Pitch response   (critical H_\\theta = %.2f m)', gust.crit.H_th), false);
xlabel(ax3, 't  [s]');
end

% ------------------------------------------------------------------------

function draw_family(ax, t, Y, show, pal, Hvec, idx_crit, ylab, ttl, do_legend)
% Same representative gusts and colours in every panel; the critical one bold black.
hold(ax, 'on');
for i = 1:numel(show)
    plot(ax, t, Y(:,show(i)), 'Color',pal(i,:), 'LineWidth',1.0, ...
        'DisplayName',sprintf('H = %.2f m', Hvec(show(i))));
end
plot(ax, t, Y(:,idx_crit), 'k', 'LineWidth',2.2, 'DisplayName','critical');
yline(ax, 0, 'Color',[.85 .85 .85], 'HandleVisibility','off');
```

```
ylabel(ax, ylab);  grid(ax, 'on');  xlim(ax, [0 max(t)]);  title(ax, ttl);
if do_legend, legend(ax, 'Location','northeast'); end
end
```

Listing 11: `impulse_all.m` — 선형 · 비선형 임펄스 응답 비교

B.5.5 `moment_response.m`

날개 루트 굽힘 모멘트 $M_b = L \cdot \int_0^L h(x)\,dx$ 의 등가 척도를 임펄스 · 멀티사인 입력 모두에 대해 선형 · 비선형 두 경우로 계산하고 출력한다.

```
function out = moment_response(L)
% moment_response  Equivalent root bending moment: impulse & multisine.
%
%   out = moment_response(L)
%       L : wing length [m]  (default 0.50)
%
%   Produces two horizontally-aligned plots.
%   Each plot shows two curves: constant k (linear) and nonlinear k (cubic).

if nargin < 1, L = 0.50; end

P     = load_params();
t_end = 8;
fs    = 2000;

% --- impulse -------------------------------------------------------
[t_i_lin, h_i_lin, th_i_lin] = linear_case(P, 1, t_end, fs);
r_imp = nonlinear_response(1);  close(gcf);

M_i_lin = equivalent_moment(h_i_lin,   th_i_lin,   P, L, false);
M_i_nl  = equivalent_moment(r_imp.h,   r_imp.theta, P, L, true);

% --- multisine -----------------------------------------------------
[t_m_lin, h_m_lin, th_m_lin] = linear_case(P, 2, t_end, fs);
r_ms = nonlinear_response(2);  close(gcf);

M_m_lin = equivalent_moment(h_m_lin, th_m_lin,  P, L, false);
M_m_nl  = equivalent_moment(r_ms.h,  r_ms.theta, P, L, true);

out = struct( ...
    'L',       L, ...
    't_i_lin', t_i_lin,  'M_i_lin', M_i_lin, ...
    't_i_nl',  r_imp.t,  'M_i_nl',  M_i_nl, ...
    't_m_lin', t_m_lin,  'M_m_lin', M_m_lin, ...
    't_m_nl',  r_ms.t,   'M_m_nl',  M_m_nl);

plot_moments(out);
end

% ========================================================================

function P = load_params()
here = fileparts(mfilename('fullpath'));
f = fullfile(here, 'wing_nl.mat');
if ~isfile(f)
    error('moment_response:model', 'wing_nl.mat not found -- run build_ss_nl.m first.');
end
P = load(f);
P.file = f;
end

% ------------------------------------------------------------------------

function [t, h, th] = linear_case(P, input_idx, t_end, fs)
% Build the same input signal as nonlinear_response/to_forcing, then lsim.
t   = (0:1/fs:t_end)';
mag = 1.0;
t0  = 0.05;

switch input_idx
    case 1                                          % impulse: short rectangular pulse
        w  = 5/fs;
        ug = mag * double(t >= t0 & t < t0 + w);

    case 2                                          % multisine (Schroeder phases)
        f0  = 0.25;  K = 80;  kk = 1:K;
        amp = 1.5 / sqrt(K/2);
        phi = -pi * kk .* (kk-1) / K;
        ug  = sum(amp .* sin(2*pi*f0*(t*kk) + phi), 2);
end

sys = ss(P.A, P.B, P.C_out, P.D_out);
y   = lsim(sys, ug, t);
h   = y(:,1);
th  = y(:,2);
end

```

```
% ------------------------------------------------------------------------

function M = equivalent_moment(h, th, P, L, nl)
d1 = h + P.r1*th;
d2 = h + P.r2*th;
if nl
    F1 = P.k1*d1 + P.knl1*d1.^3;
    F2 = P.k2*d2 + P.knl2*d2.^3;
else
    F1 = P.k1*d1;
    F2 = P.k2*d2;
end
Mb = L*(F1 + F2);
T  = P.r1*F1 + P.r2*F2;
M  = 0.5*( abs(Mb) + sqrt(Mb.^2 + T.^2) );
end

% ------------------------------------------------------------------------

function plot_moments(out)
col_lin  = [0.20 0.40 0.80];   % blue   -- constant k
col_nl   = [0.85 0.33 0.10];   % orange -- nonlinear k
col_diff = [0.30 0.60 0.30];   % green  -- absolute difference
labels   = {'Constant k', 'Nonlinear k', '|Difference|'};

% Interpolate nonlinear (ode45 adaptive grid) onto the uniform linear grid
diff_i = abs(out.M_i_lin - interp1(out.t_i_nl, out.M_i_nl, out.t_i_lin, 'linear', 'extrap'));
diff_m = abs(out.M_m_lin - interp1(out.t_m_nl, out.M_m_nl, out.t_m_lin, 'linear', 'extrap'));

figure('Name','Equivalent bending moment','Position',[100 100 1100 420]);
tiledlayout(1,2,'TileSpacing','compact','Padding','compact');

nexttile
plot(out.t_i_lin, out.M_i_lin, 'Color',col_lin,  'LineWidth',1.5)
hold on
plot(out.t_i_nl,  out.M_i_nl,  'Color',col_nl,   'LineWidth',1.5)
plot(out.t_i_lin, diff_i,      'Color',col_diff, 'LineWidth',1.2, 'LineStyle','--')
yline(0,'Color',[.7 .7 .7],'HandleVisibility','off');  grid on
xlabel('t  [s]');  ylabel('M_{eq}  [N m]')
title(sprintf('Impulse response   (L = %.2f m)', out.L))
legend(labels, 'Location','northeast')

nexttile
plot(out.t_m_lin, out.M_m_lin, 'Color',col_lin,  'LineWidth',1.5)
hold on
plot(out.t_m_nl,  out.M_m_nl,  'Color',col_nl,   'LineWidth',1.5)
plot(out.t_m_lin, diff_m,      'Color',col_diff, 'LineWidth',1.2, 'LineStyle','--')
yline(0,'Color',[.7 .7 .7],'HandleVisibility','off');  grid on
xlabel('t  [s]');  ylabel('M_{eq}  [N m]')
title('Multisine response')
legend(labels, 'Location','northeast')

here = fileparts(mfilename('fullpath'));
exportgraphics(gcf, fullfile(here, 'nl_moment_response.png'), 'Resolution', 300);
fprintf('Saved: nl_moment_response.png\n');
end
```

Listing 12: `moment_response.m` — 등가 루트 굽힘 모멘트

B.5.6 `result_plot.m`

4주차 해석 전체를 요약하는 5 × 2 서브플롯 종합 그래프를 생성한다. 임펄스 · 멀티사인 각 입력에 대해 플런지, 피치, 등가 굽힘 모멘트를 열로 배치하고 선형(파란색) · 비선형(주황색) 응답을 겹쳐 표시한다.

```
function result_plot()
% result_plot  5x2 summary figure for Week-4 nonlinear wing analysis.
%
%   Layout
%   -------
%   Col 1 (linear / constant-k)              | Col 2 (nonlinear, gust option 4)
%   Row 1  gust family (impulse_fg sp1)      | modified gust input u(t)
%   Row 2  tuning sweep (impulse_fg sp2)     | plunge h(t)
%   Row 3  critical response (impulse_fg sp3)| pitch theta(t)
%   Row 4  spring-1 constant k               | spring-1 tangent k1(t)
%   Row 5  equiv. bending moment, const-k    | equiv. bending moment, varying-k

here = fileparts(mfilename('fullpath'));

% ---- 1. Run source functions (suppress internal figures) -------------
gust = impulse_function_generator('plot', false);

nl = nonlinear_response(4);
close(gcf);

mout = moment_response();
close(gcf);

% ---- 2. Linear critical response (for rows 3 and 4) ------------------
mdl     = load(fullfile(here, '..', 'week3', 'wing_2dof.mat'), ...
```

```
                'A','B','C_out','D_out');
sys_lin = ss(mdl.A, mdl.B, mdl.C_out, mdl.D_out);
wg_crit = gust.wg(:, gust.crit.idx_h);
y_lin   = lsim(sys_lin, wg_crit, gust.t);   % columns: h [m], theta [rad]

% constant spring-1 stiffness (linear assumption: k1 independent of deflection)
nlp      = load(fullfile(here, 'wing_nl.mat'), 'k1');
k1_const = nlp.k1 * ones(size(gust.t));

% ---- 3. Colours (shared with source scripts) -------------------------
col_h  = [0.85 0.33 0.10];
col_th = [0.20 0.40 0.80];
col_k  = [0.30 0.60 0.30];

% ---- 4. Figure -------------------------------------------------------
figure('Name','Week-4 Results','Position',[80 60 1200 1100]);

% -- Row 1 (full width): 1-cosine gust family --------------------------
subplot(5,2,[1 2])
show = unique(round(linspace(1, numel(gust.H), 5)));
hold on
for j = show
    plot(gust.t, gust.wg(:,j), 'LineWidth',1.1, ...
         'DisplayName', sprintf('H = %.1f m', gust.H(j)));
end
plot(gust.t, gust.wg(:,gust.crit.idx_h), 'k', 'LineWidth',2, ...
     'DisplayName','critical H');
hold off
xlabel('t [s]');  ylabel('w_g [m/s]');  grid on
title('1-cosine gust family  (U_d_s/2)[1-cos(\pi s/H)]')
legend('Location','northeast')
xlim([0 max(gust.t)])

% -- Row 2, Col 1: gradient-tuning sweep -------------------------------
subplot(5,2,3)
yyaxis left
plot(gust.H, gust.resp.h_peak, '-o', 'Color',col_h, 'MarkerSize',3, ...
     'DisplayName','peak |h|');
ylabel('peak |h| [mm]')
yyaxis right
plot(gust.H, gust.resp.th_peak, '-s', 'Color',col_th, 'MarkerSize',3, ...
     'DisplayName','peak |\theta|');
ylabel('peak |\theta| [deg]')
xlabel('H [m]');  grid on
xline(gust.crit.H_h,  '--', 'Color',col_h,  'HandleVisibility','off')
xline(gust.crit.H_th, '--', 'Color',col_th, 'HandleVisibility','off')
title(sprintf('Tuning sweep:  H_h = %.2f m,  H_\\theta = %.2f m', ...
              gust.crit.H_h, gust.crit.H_th))
legend('Location','northeast')

% -- Row 2, Col 2: nonlinear h(t) --------------------------------------
subplot(5,2,4)
plot(nl.t, nl.h*1000, 'Color',col_h, 'LineWidth',1.5, 'DisplayName','h(t)')
yline(0,'Color',[.7 .7 .7],'HandleVisibility','off');  grid on
xlabel('t [s]');  ylabel('h [mm]')
title('Plunge h(t)  (nonlinear gust response)')
legend('Location','northeast')

% -- Row 3, Col 1: linear critical response (h and theta) --------------
subplot(5,2,5)
yyaxis left
plot(gust.t, y_lin(:,1)*1000, 'Color',col_h, 'LineWidth',1.5, ...
     'DisplayName','h');
ylabel('h [mm]')
yyaxis right
plot(gust.t, rad2deg(y_lin(:,2)), 'Color',col_th, 'LineWidth',1.5, ...
     'DisplayName','\theta');
ylabel('\theta [deg]')
xlabel('t [s]');  grid on
title('Linear response to critical gust')
legend('Location','northeast')

% -- Row 3, Col 2: nonlinear theta(t) ----------------------------------
subplot(5,2,6)
plot(nl.t, rad2deg(nl.theta), 'Color',col_th, 'LineWidth',1.5, ...
     'DisplayName','\theta(t)')
yline(0,'Color',[.7 .7 .7],'HandleVisibility','off');  grid on
xlabel('t [s]');  ylabel('\theta [deg]')
title('Pitch \theta(t)  (nonlinear gust response)')
legend('Location','northeast')

% -- Row 4, Col 1: constant spring stiffness (linear assumption) -------
subplot(5,2,7)
plot(gust.t, k1_const, 'Color',col_k, 'LineWidth',1.5, ...
     'DisplayName','k_1 (const)')
xlabel('t [s]');  ylabel('k_1 [N/m]');  grid on
title('Spring-1 stiffness  (linear: k_1 = const)')
legend('Location','northeast')

% -- Row 4, Col 2: nonlinear tangent stiffness k1eff(t) ----------------
subplot(5,2,8)
plot(nl.t, nl.k1eff, 'Color',col_k, 'LineWidth',1.5, ...
```

```
     'DisplayName','k_1(t)')
xlabel('t [s]');  ylabel('k_1(t) [N/m]');  grid on
title('Spring-1 tangent stiffness  k_1 + 3k_n_l_1\delta_1^2')
legend('Location','northeast')

% -- Row 5, Col 1: equivalent bending moment - constant k --------------
subplot(5,2,9)
plot(mout.t_lin, mout.M_lin, 'Color',col_h, 'LineWidth',1.5, ...
     'DisplayName','M_e_q')
yline(0,'Color',[.7 .7 .7],'HandleVisibility','off');  grid on
xlabel('t [s]');  ylabel('M_e_q [N m]')
title(sprintf('Equiv. bending moment - constant k  (L = %.2f m)', mout.L))
legend('Location','northeast')

% -- Row 5, Col 2: equivalent bending moment - varying k ---------------
subplot(5,2,10)
plot(mout.t_nl, mout.M_nl, 'Color',col_h, 'LineWidth',1.5, ...
     'DisplayName','M_e_q')
yline(0,'Color',[.7 .7 .7],'HandleVisibility','off');  grid on
xlabel('t [s]');  ylabel('M_e_q [N m]')
title('Equiv. bending moment - varying k  (cubic springs)')
legend('Location','northeast')

end
```

Listing 13: `result_plot.m` — 4주차 종합 결과 그래프

B.6 5주차: 속도에 따른 고유값 변화 및 플러터 경계 (MATLAB)

비행속도 U가 변함에 따라 시스템 행렬 $\mathbf{A}(U)$의 고유값이 이동하는 양상을 root-locus로 추적하고, 무게중심–공력중심 간격 e_a에 따른 플러터 경계를 2D/3D로 시각화한다.

B.6.1 `root_locus_U.m`

속도 U를 근-영에서 최대값까지 스윕하면서 각 U에서 $\mathbf{A}(U)$의 고유값을 계산하여 복소평면에 궤적을 그린다. 허수축 (Re = 0)을 넘는 임계 속도가 플러터 속도 U_f로 표시된다.

```
function root_locus_U()
% Root locus of the 2-DOF aeroelastic wing as aero.U varies.
%
% Sweeps freestream speed U from near-zero to U_max, builds the linear
% A-matrix at each U (same parameters as build_2dof in build_models.m),
% tracks the four eigenvalue branches, locates the flutter speed where
% a branch crosses the imaginary axis (Re = 0, Im ≠ 0), and saves a plot.

% ---- physical parameters (must match build_2dof / aero_terms) ----------
m  = 1.5;      Ip = 0.0084;     S  = 0.0225;
k  = 5500.0;   cd = 1.0;      d  = 0.22;

rho      = 1.225;    chord    = 0.30;
span     = 0.50;     Clalpha = 6;
ea       = 0.045;
U_nom    = 12.0;     % nominal speed from aero_terms [m/s]

M_mat    = [m,    S;    S,   Ip];
Ks       = [2*k,  0;    0,   k*d^2/2];
C_struct = [2*cd, 0;    0,   cd*d^2/2];

% ---- velocity sweep -----------------------------------------------------
U_vec = linspace(0.5, 50, 4000);
N     = numel(U_vec);

ev_mat = zeros(4, N);   % each column: eigenvalues sorted by imag part

for i = 1:N
    U  = U_vec(i);
    qS = 0.5 * rho * U^2 * chord * span;

    Ka   = [0,  qS*Clalpha;   0, -qS*Clalpha*ea];
    Ca   = (qS*Clalpha / U) .* [1, 0;  -ea, 0];

    Minv = inv(M_mat);
    A    = [zeros(2),         eye(2);
            -Minv*(Ks+Ka),  -Minv*(C_struct+Ca)];

    ev = eig(A);
    [~, idx] = sort(imag(ev));   % ascending imag → consistent branch order
    ev_mat(:, i) = ev(idx);
end

% ---- find flutter speed: first U where max Re crosses 0 from below ----
max_re = max(real(ev_mat), [], 1);
cross  = find(diff(sign(max_re)) > 0, 1);

U_fl  = NaN;
im_fl = NaN;
```

```
if ~isempty(cross)
    % linear interpolation between the two straddle points
    U_fl = interp1(max_re(cross:cross+1), U_vec(cross:cross+1), 0);

    % find which branch is the one crossing
    for b = 1:4
        re_b = real(ev_mat(b, :));
        if re_b(cross) < 0 && re_b(cross+1) >= 0
            im_fl = interp1(U_vec(cross:cross+1), ...
                            imag(ev_mat(b, cross:cross+1)), U_fl);
            break
        end
    end

    fprintf('Flutter speed:     U_flutter = %.4f m/s\n', U_fl);
    fprintf('Flutter frequency: %.4f rad/s  (%.4f Hz)\n', ...
            abs(im_fl), abs(im_fl)/(2*pi));
else
    fprintf('No flutter detected in U = [%.1f, %.1f] m/s.\n', ...
            U_vec(1), U_vec(end));
end

% ---- plot ------------------------------------------------------------
figure('Name', 'Root Locus vs aero.U', 'Position', [80 80 950 680]);
hold on;  grid on;  ax = gca;

colors = lines(4);
branch_labels = {'Mode 1 (lower \omega)','Mode 2 (upper \omega)',...
                 'Mode 2 (conj.)','Mode 1 (conj.)'};

for b = 1:4

plot(real(ev_mat(b,:)), imag(ev_mat(b,:)), '-', ...
         'Color', colors(b,:), 'LineWidth', 1.8, ...
         'DisplayName', branch_labels{b});
      %{
    plot(real(ev_mat(b,:)), imag(ev_mat(b,:)), '-', ...
         'Color', colors(b,:), 'LineWidth', 1.8);
      %}
    % circle at low-U end
    plot(real(ev_mat(b,1)), imag(ev_mat(b,1)), 'o', ...
         'Color', colors(b,:), 'MarkerFaceColor', colors(b,:), ...
         'MarkerSize', 6, 'HandleVisibility', 'off');

    % arrow mid-branch to show direction of increasing U
    mid = round(N/2);
    quiver(real(ev_mat(b, mid)), imag(ev_mat(b, mid)), ...
           real(ev_mat(b, mid+1)) - ev_mat(b, mid)), ...
           imag(ev_mat(b, mid+1) - ev_mat(b, mid)), ...
           5, 'Color', colors(b,:), 'MaxHeadSize', 3, ...
           'HandleVisibility', 'off');
end

% imaginary axis
xline(0, 'k--', 'LineWidth', 1.4, 'HandleVisibility', 'off');
text(0.5, ax.YLim(1)*0.92, 'Im axis', 'FontSize', 9, 'Color', [0.3 0.3 0.3]);

%{
% nominal operating point (U = 12 m/s)
nom_idx = round(interp1(U_vec, 1:N, U_nom));
for b = 1:4
    plot(real(ev_mat(b, nom_idx)), imag(ev_mat(b, nom_idx)), 'ks', ...
         'MarkerSize', 8, 'LineWidth', 1.5, 'HandleVisibility', 'off');
end
% one legend entry for the nominal marker
plot(nan, nan, 'ks', 'MarkerSize', 8, 'LineWidth', 1.5, ...
     'DisplayName', sprintf('Nominal  U = %.0f m/s', U_nom));
%}

% flutter marker
if ~isnan(U_fl)
    plot(0,  im_fl, 'rp', 'MarkerSize', 18, 'MarkerFaceColor', 'r', ...
         'LineWidth', 1.2, ...
         'DisplayName', sprintf('Flutter  U_{fl} = %.2f m/s', U_fl));
    plot(0, -im_fl, 'rp', 'MarkerSize', 18, 'MarkerFaceColor', 'r', ...
         'HandleVisibility', 'off');
    text(0.4,  im_fl, sprintf('  U_{fl} = %.2f m/s', U_fl), ...
         'FontSize', 11, 'Color', 'r', 'FontWeight', 'bold');
    text(0.4, -im_fl, sprintf('  U_{fl} = %.2f m/s', U_fl), ...
         'FontSize', 11, 'Color', 'r', 'FontWeight', 'bold');
end

xlabel('Real part  \sigma  [1/s]',          'FontSize', 12);
ylabel('Imaginary part  \omega_d  [rad/s]', 'FontSize', 12);
title({'Root Locus w.r.t. aero.U  --  2-DOF aeroelastic wing'}, ...
      'FontSize', 12);
legend('Location', 'best', 'FontSize', 9);

here = fileparts(mfilename('fullpath'));
out  = fullfile(here, 'root_locus_U.png');
%exportgraphics(gcf, out, 'Resolution', 300);
fprintf('Figure saved: %s\n', out);
```

```
end
```

Listing 14: `root_locus_U.m` — 속도에 따른 root-locus

B.6.2 `flutter_vs_d.m`

e_a 값을 변화시키면서 각각의 플러터 속도 U_f를 계산하여 2D 선 그래프와 3D 안정성 곡면(max Re(λ) vs (e_a,U))을 생성한다. 플러터 경계를 최고 비행속도 이상으로 확보하기 위한 최적 e_a 설계 근거를 제공한다.

```
%function [d_cm,U_fl] = flutter_vs_d()
function flutter_vs_d()
% Flutter speed and stability surface as a function of CG-to-AC distance d.
%
% d = x_CG - x_AC  [m]:  + when CG is aft of AC (destabilising)
%                        - when CG is forward of AC  (stabilising)
%
% NACA 2412 context: AC at ~25% chord; user specifies CG also at 25% chord
% => d_nom = 0, which maps to S = m*(0 - ea) = -m*ea = -0.10 kg.m.
%
% Relation to build_2dof parameters (build_models.m):
%   S      = m * (d - ea)        static unbalance [kg.m]
%   M_mat = [m, S; S, Ip]        mass matrix
%
% Physical constraint for PD mass matrix:
%   m*Ip > S^2  =>  |d - ea| < sqrt(Ip/m) = r_gyr
%
% Outputs:
%   Figure 1  2-D line:  U_flutter vs d
%   Figure 2  3-D surf:  max Re(lambda) over (d, U) space
%             Red curve on surface = flutter boundary (Re = 0)

% ---- fixed parameters (match build_2dof / aero_terms in build_models.m) --
m  = 1.5;      Ip = 0.0084;
k  = 5500.0;   cd = 1.0;      d_sp  = 0.22;  % d_sp = spring spacing (not CG-AC d!)

rho = 1.225;   chord = 0.30;  span = 0.50;
Clalpha = 6;   ea = 0.045;             % AC forward of elastic axis [m]

% Structural matrices (independent of d or U)
Ks       = [2*k,  0;   0, k*d_sp^2/2];
C_struct = [2*cd, 0;   0, cd*d_sp^2/2];

% ---- parametric range -----------------------------------------------------
% PD constraint: |d - ea| < sqrt(Ip/m)
r_gyr = sqrt(Ip / m);             % = 0.0707 m
d_min =  ea - r_gyr + 5e-4;      % m  (add small margin)
d_max =  ea + r_gyr - 5e-4;

d_vec = linspace(d_min, d_max, 200);   % CG-to-AC distance [m]
U_vec = linspace(0.5,   50,   500);    % freestream speed  [m/s]

Nd = numel(d_vec);
Nu = numel(U_vec);

max_re_surf = nan(Nu, Nd);   % max Re(eig(A)) at each (U, d) point
U_fl        = nan(1,  Nd);   % flutter speed for each d

for id = 1:Nd
    d_cga = d_vec(id);
    S     = m * (d_cga - ea);    % static unbalance
    M     = [m, S;  S, Ip];

    if det(M) <= 0; continue; end
    Minv = inv(M);

    for iu = 1:Nu
        U  = U_vec(iu);
        qS = 0.5 * rho * U^2 * chord * span;

        Ka = [0,  qS*Clalpha;   0, -qS*Clalpha*ea];
        Ca = (qS*Clalpha / U) .* [1, 0;  -ea, 0];

        A  = [zeros(2),          eye(2);
             -Minv*(Ks + Ka),  -Minv*(C_struct + Ca)];

        max_re_surf(iu, id) = max(real(eig(A)));
    end

    % flutter crossing: first U where max Re crosses 0 from below
    col = max_re_surf(:, id)';
    cx  = find(diff(sign(col)) > 0, 1);
    if ~isempty(cx)
        U_fl(id) = interp1(col(cx:cx+1), U_vec(cx:cx+1), 0);
    end
end

d_cm = d_vec * 100;   % display in cm

% print summary
```

```
fprintf('CG-to-AC distance range: [%.2f, %.2f] cm\n', d_cm(1), d_cm(end));
fprintf('d = 0 (CG at AC, NACA2412 nom.): U_flutter = %.3f m/s\n', ...
    interp1(d_cm, U_fl, 0, 'linear', NaN));
[min_Ufl, i_min] = min(U_fl);
fprintf('Lowest flutter:  U_fl = %.3f m/s at d = %.2f cm\n', min_Ufl, d_cm(i_min));

% ========================================================================
% Figure 1 -- 2-D:  U_flutter vs d
% ========================================================================
f1 = figure('Name','Flutter speed vs d', 'Position',[60 80 760 470]);
plot(d_cm, U_fl, 'b-', 'LineWidth', 2.5);
hold on;  grid on;

% mark d = 0 (CG at AC)
xline(0, 'k--', 'LineWidth', 1.3, 'HandleVisibility','off');
text(0.3, min(U_fl, [], 'omitnan') * 0.93, ...
    '  d = 0  (CG at AC)', 'FontSize', 9, 'Color', [0.3 0.3 0.3]);

% mark original model point (d from original S = 0.015)
S_orig   = 0.015;
d_orig   = S_orig/m + ea;   % = 0.0575 m = 5.75 cm
Ufl_orig = interp1(d_cm, U_fl, d_orig*100, 'linear', NaN);
if ~isnan(Ufl_orig)
    plot(d_orig*100, Ufl_orig, 'rs', 'MarkerSize', 10, 'MarkerFaceColor','r', ...
        'DisplayName', sprintf('build\\_2dof  d=%.1fcm, U_{fl}=%.1f m/s', ...
        d_orig*100, Ufl_orig));
    legend('Location','best','FontSize',9);
end

xlabel('d = x_{CG} - x_{AC}  [cm]     (+ : CG aft of AC)', 'FontSize', 12);
ylabel('Flutter speed  U_{flutter}  [m/s]', 'FontSize', 12);
title({'Flutter speed vs CG-to-AC distance  d', ...
    'NACA 2412  --  CG at 25% chord when d = 0'}, 'FontSize', 12);

% ========================================================================
% Figure 2 -- 3-D surf:  max Re(lambda) over (d, U)
% ========================================================================
[D_cm, UU] = meshgrid(d_cm, U_vec);

f2 = figure('Name','3D stability surface (d, U)', 'Position',[130 80 1000 680]);

% surface
surf(D_cm, UU, max_re_surf, 'EdgeColor','none', 'FaceAlpha', 0.88);
hold on;

% flutter boundary: Re = 0 contour on the surface (red)
contour3(D_cm, UU, max_re_surf, [0 0], 'r-', 'LineWidth', 3.5);

% transparent Re = 0 plane for reference
xlm = [min(d_cm), max(d_cm)];
ylm = [U_vec(1),  U_vec(end)];
patch([xlm(1) xlm(2) xlm(2) xlm(1)], ...
    [ylm(1) ylm(1) ylm(2) ylm(2)], ...
    [0 0 0 0], [0.6 0.6 0.6], ...
    'FaceAlpha', 0.12, 'EdgeColor','none');

% original model marker projected onto surface
if ~isnan(Ufl_orig) && d_orig*100 >= d_cm(1) && d_orig*100 <= d_cm(end)
    z_marker = interp2(D_cm, UU, max_re_surf, d_orig*100, Ufl_orig, 'linear', NaN);
    plot3(d_orig*100, Ufl_orig, 0, 'rs', 'MarkerSize', 12, ...
        'MarkerFaceColor','r', 'LineWidth', 1.5);
end

% colormap: blue (stable) -> white (neutral) -> red (unstable)
colormap(bwr_cmap(256));
c_lim = prctile(abs(max_re_surf(:)), 95);
clim([-c_lim, c_lim]);
cb = colorbar;
cb.Label.String  = 'max  Re(\lambda)  [1/s]';
cb.Label.FontSize = 11;

xlabel('d = x_{CG} - x_{AC}  [cm]', 'FontSize', 11);
ylabel('Freestream speed  U  [m/s]', 'FontSize', 11);
zlabel('max  Re(\lambda)  [1/s]',    'FontSize', 11);
title({'Stability surface over (CG-to-AC distance  d,  speed  U)', ...
    'Red curve = flutter boundary  [Re(\lambda) = 0]'}, 'FontSize', 12);
view([-42, 28]);

% ---- save ------------------------------------------------------------
%here = fileparts(mfilename('fullpath'));
%exportgraphics(f1, fullfile(here, 'flutter_vs_d_2D.png'),'Resolution', 300);
%exportgraphics(f2, fullfile(here, 'flutter_vs_d_3D.png'),'Resolution', 300);
%fprintf('Saved: flutter_vs_d_2D.png  and  flutter_vs_d_3D.png\n');
end

% ========================================================================
% Local helper: diverging blue-white-red colormap
% ========================================================================
function cmap = bwr_cmap(n)
n2 = ceil(n / 2);
blue_to_white = [linspace(0,1,n2)', linspace(0,1,n2)', ones(n2,1)];
white_to_red  = [ones(n2,1), linspace(1,0,n2)', linspace(1,0,n2)'];
```

```
cmap = [blue_to_white; white_to_red];
cmap = cmap(1:n, :);
end
```

Listing 15: `flutter_vs_d.m` — 간격 e_a에 따른 플러터 경계

References

[1] J.D. Anderson and C.P. Cadou. *Fundamentals of Aerodynamics*. McGraw-Hill series in aeronautical and aerospace engineering. McGraw Hill, 2023. ISBN: 9781266076442. URL: https://books.google.co.kr/books?id=PUV7zwEACAAJ.

[2] D.H. Hodges and G.A. Pierce. *Introduction to Structural Dynamics and Aeroelasticity*. Cambridge Aerospace Series. Cambridge University Press, 2011. ISBN: 9781139499927. URL: https://books.google.co.kr/books?id=_UCVJ6RY-vsC.

[3] AS Mendes, PS Meirelles, and DE Zampieri. "Analysis of torsional vibration in internal combustion engines: modelling and experimental validation". In: *Proceedings of the Institution of Mechanical Engineers, Part K: Journal of Multi-body Dynamics* 222.2 (2008), pp. 155–178.

[4] Brian L. Stevens, Frank L. Lewis, and Eric N. Johnson. "The Kinematics and Dynamics of Aircraft Motion". In: *Aircraft Control and Simulation: Dynamics, Controls Design, and Autonomous Systems*. John Wiley & Sons, Ltd, 2015. Chap. 1, pp. 1–62. ISBN: 9781119174882. DOI: https://doi.org/10.1002/9781119174882.ch1. eprint: https://onlinelibrary.wiley.com/doi/pdf/10.1002/9781119174882.ch1. URL: https://onlinelibrary.wiley.com/doi/abs/10.1002/9781119174882.ch1.